BIT.MLMR

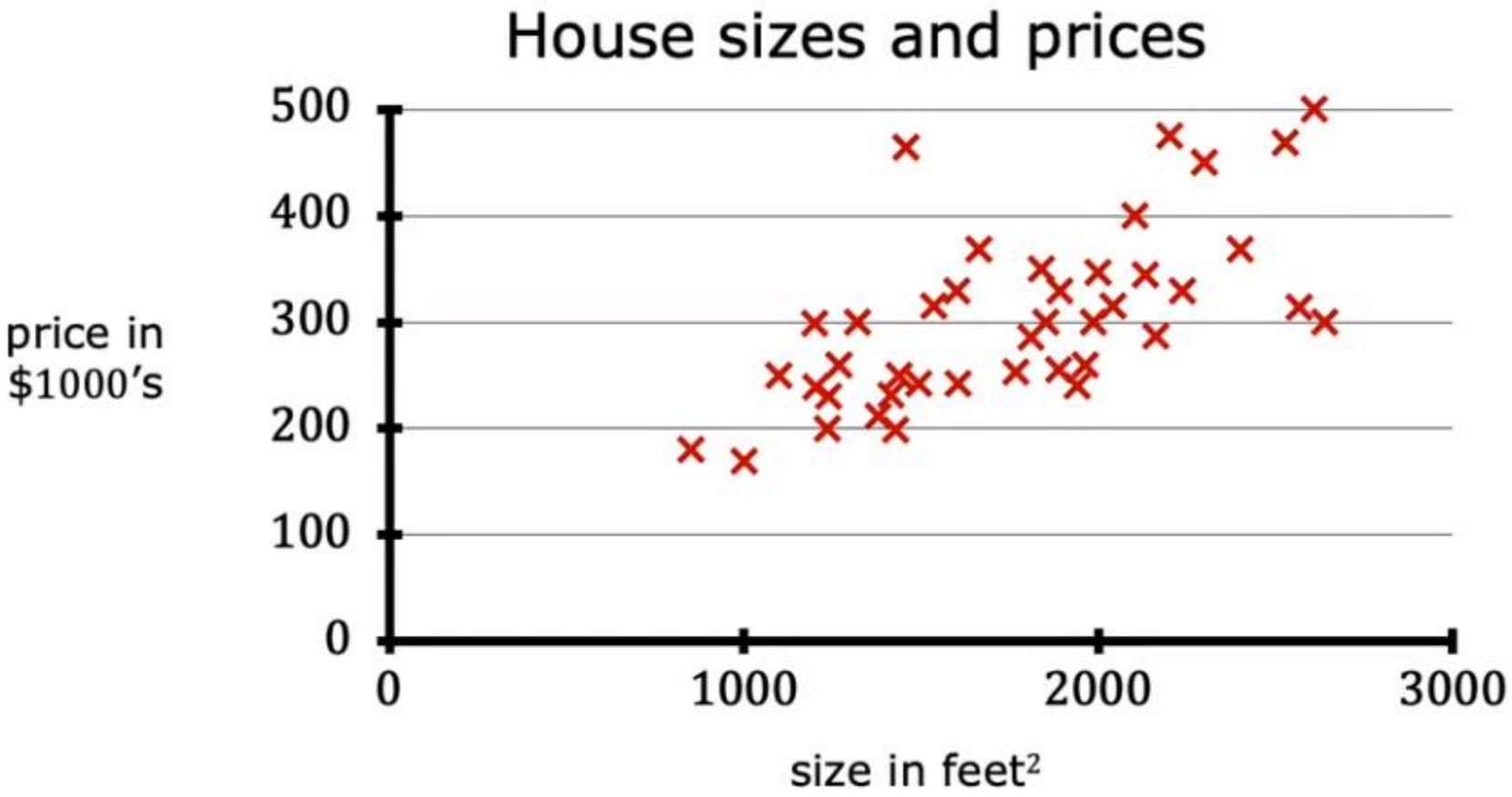
Machine Learning

Lecture 2

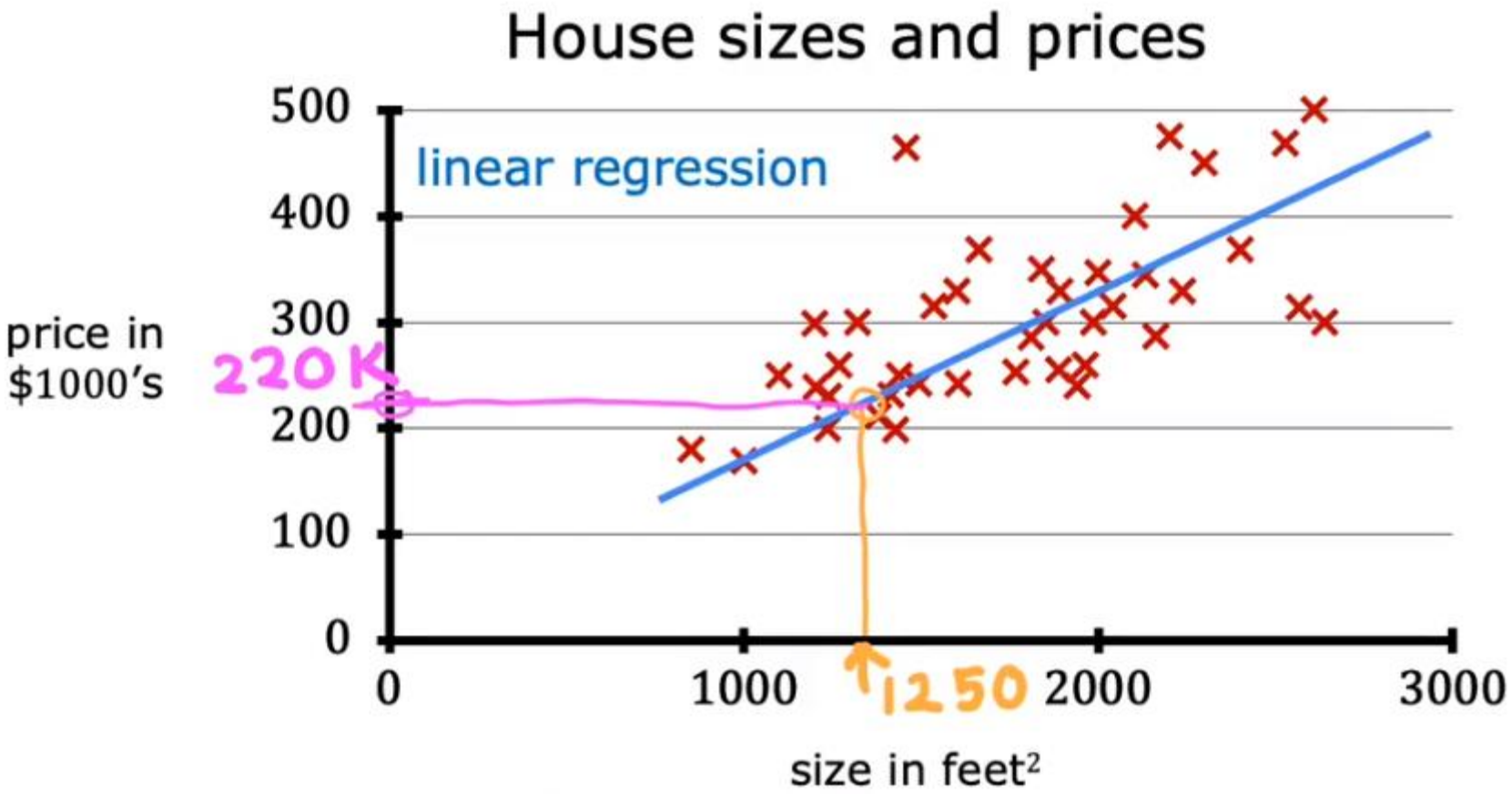
Fritz Perls: Learning is the discovery that something is possible

Herbert Simon: Learning denotes change

1. Regression



1. Regression



1. Regression

Data table

size in feet²	price in $1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870

1. Regression

Terminology

Training set: Data used to train the model

size in feet^2	price in \$1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870

1. Regression

Terminology

Training set: Data used to train the model

	x size in feet^2	y price in \$1000's
(1)	2104	400
(2)	1416	232
(3)	1534	315
(4)	852	178
...	...	...
(47)	3210	870

$m = 47$

$x = 2104$ $y = 400$

$(x, y) = (2104, 400)$

Notation:

x = "input" variable
feature

y = "output" variable
"target" variable

m = number of training examples

(x, y) = single training example

1. Regression

Terminology

Training set: Data used to train the model

	x size in feet^2	y price in \$1000's
(1)	2104	400
(2)	1416	232
(3)	1534	315
(4)	852	178
...	...	...
(47)	3210	870

$m = 47$

$x = 2104$ $y = 400$

$(x, y) = (2104, 400)$

Notation:

x = "input" variable
feature

y = "output" variable
"target" variable

m = number of training examples

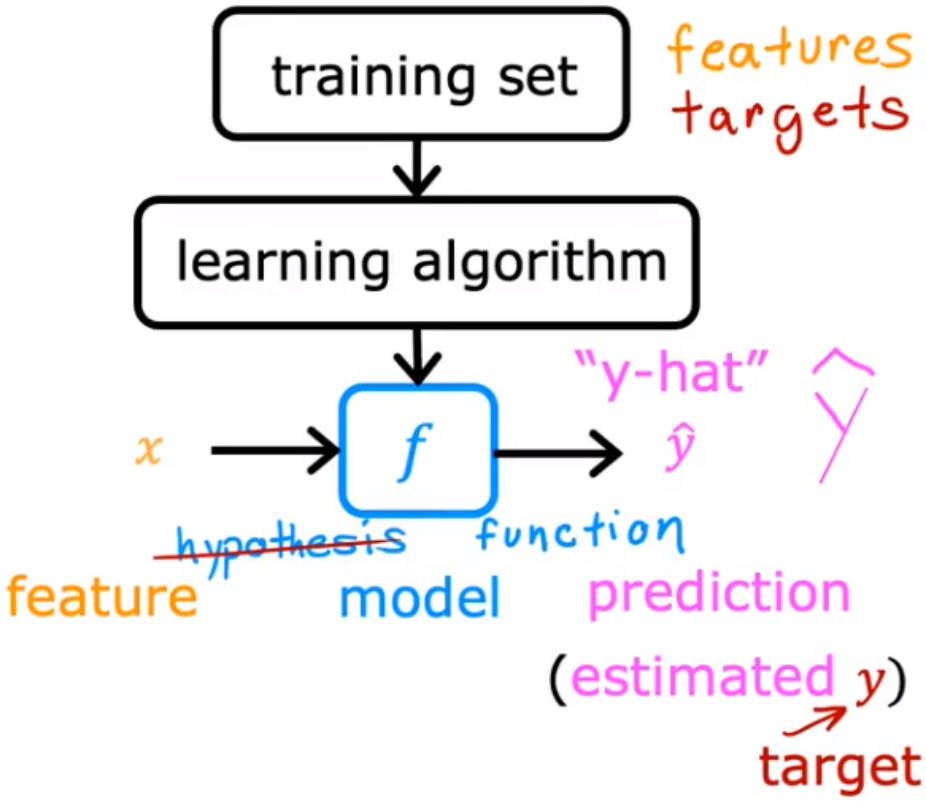
(x, y) = single training example

$(x^{(i)}, y^{(i)})$

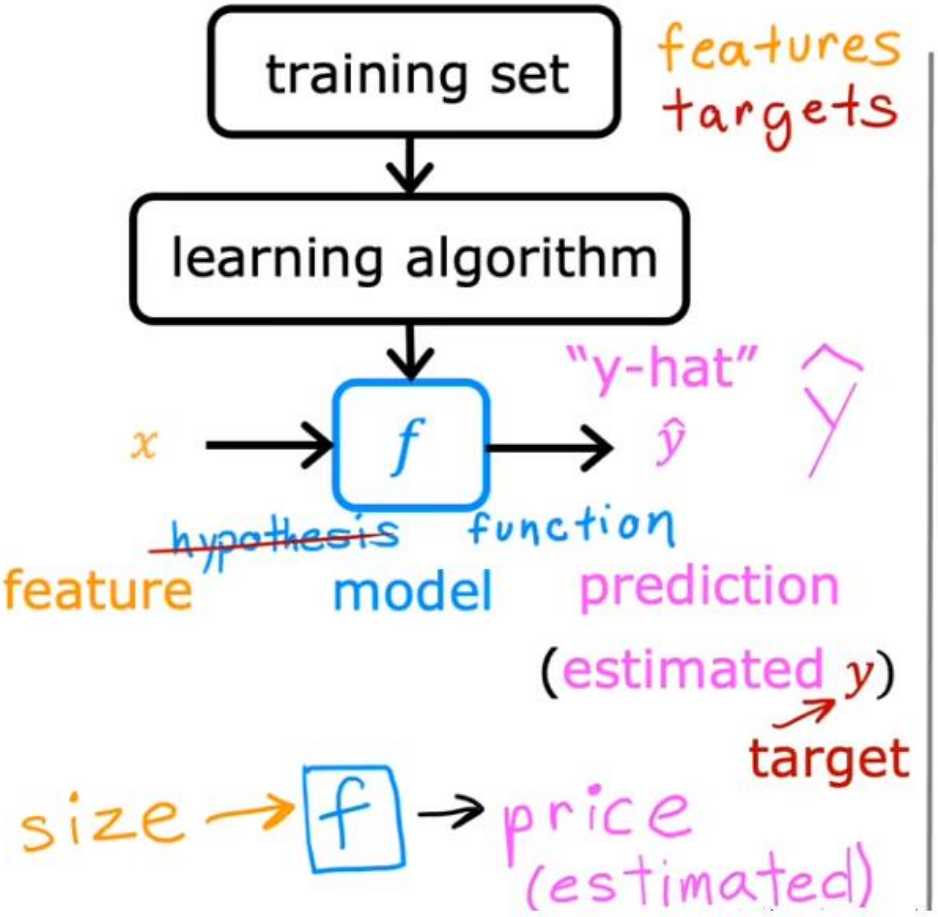
$(x^{(i)}, y^{(i)})$ = i^{th} training example

(1^{st}, 2^{nd}, 3^{rd} ...)

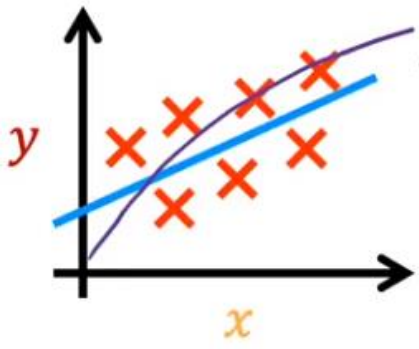
1. Regression



1. Regression



How to represent f?



$f_{w,b}(x) = wx + b$

$f(x) = wx + b$

linear

single feature x

Linear regression with one variable.

size

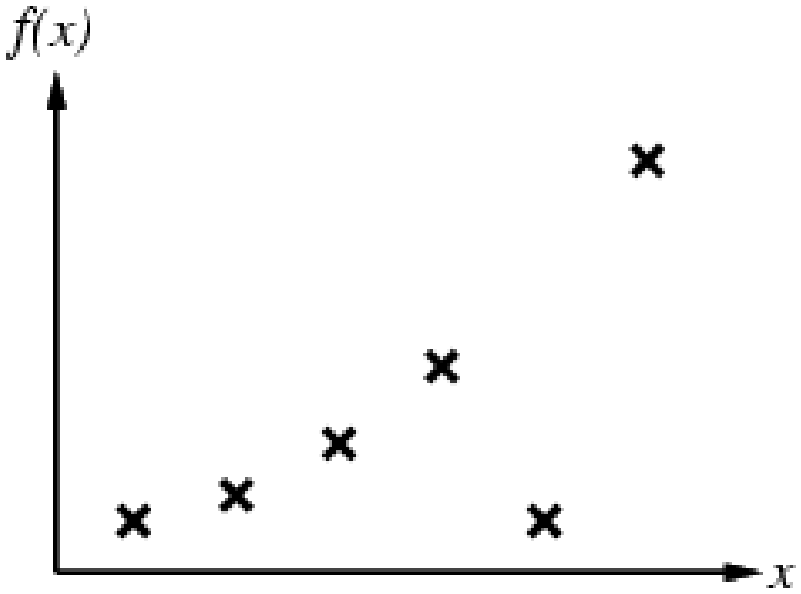
Univariate linear regression.

Supervised learning principle

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**

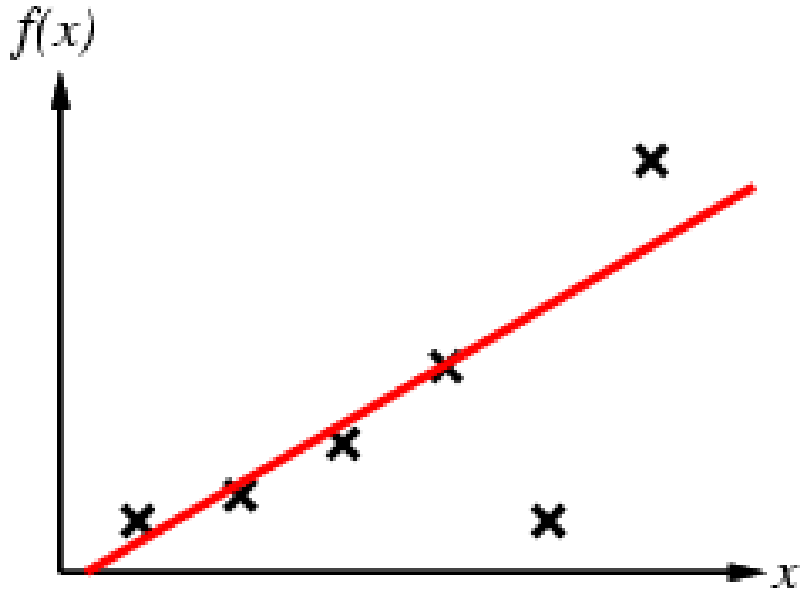


Supervised learning principle

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



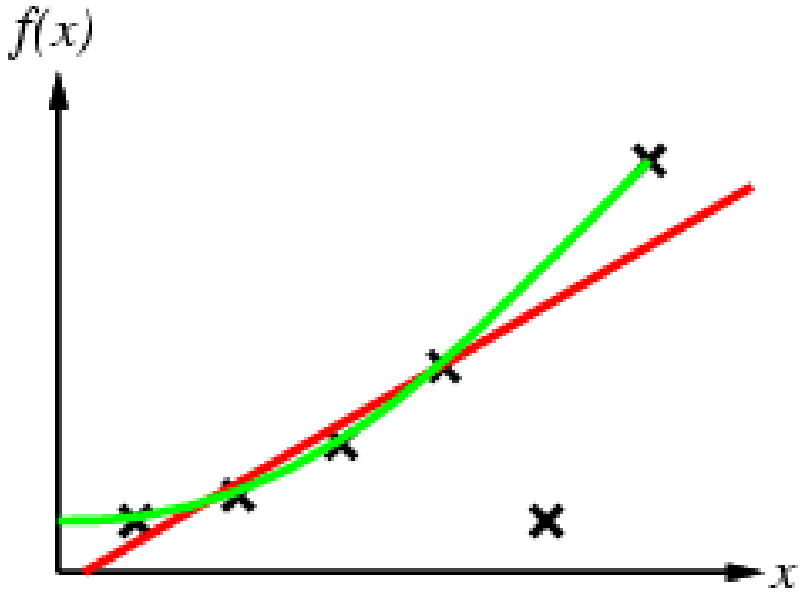
h_1: $y = ax + b$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

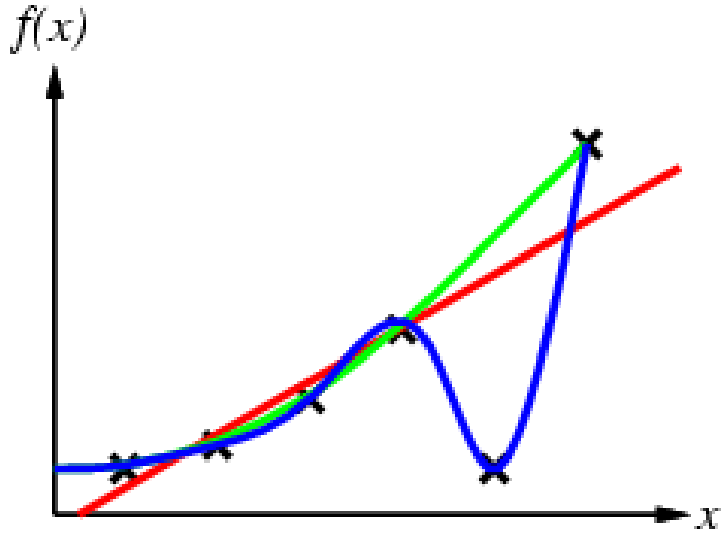
h_2: $y = ax^2 + bx + c$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

h_2: $y = ax^2 + bx + c$

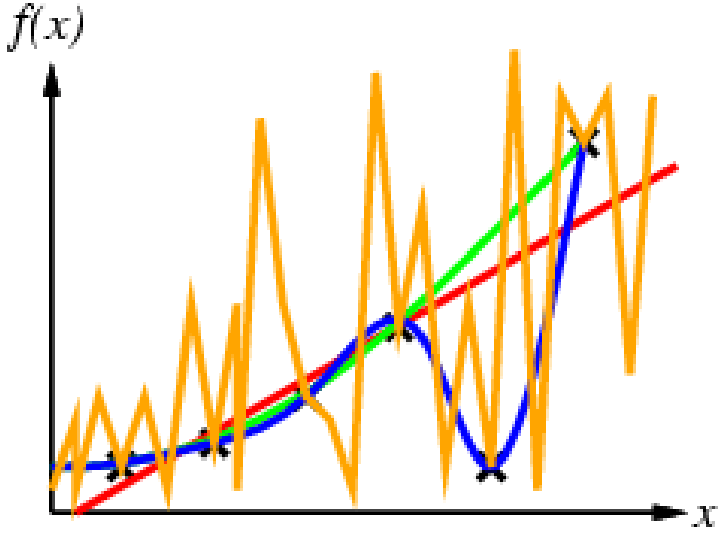
h_3: $y = ax^4 + bx^2 + cx + d$

Supervised learning method

- **Construct/adjust *h* to agree with *f* on training set**

  (*h* is consistent if it agrees with *f* on all examples)

- **E.g., curve fitting:**



h_1: $y = ax + b$

h_2: $y = ax^2 + bx + c$

h_3: $y = ax^4 + bx^2 + cx + d$

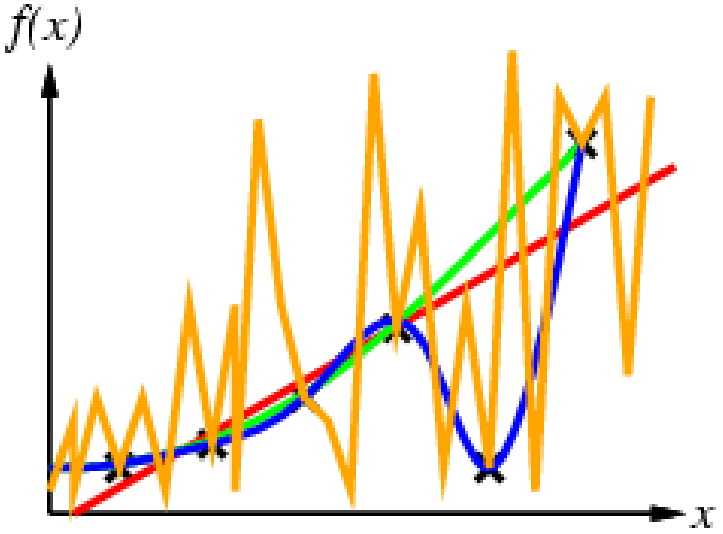
h_4: *too complicated*

Supervised learning method

- Construct/adjust *h* to agree with *f* on training set

  (*h* is consistent if it agrees with *f* on all examples)

- E.g., curve fitting:



- Ockham's razor: prefer the simplest hypothesis consistent with data

1. Regression – cost function

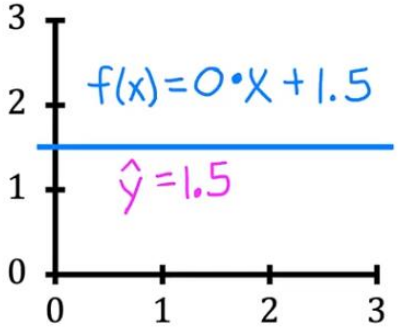
Training set

features: size in feet² (x) | targets: price \$1000's (y)

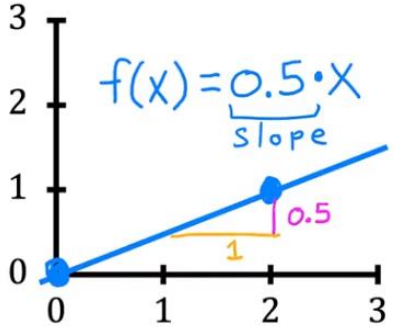
size in feet² (x)	price \$1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Model: $f_{w,b}(x) = wx + b$

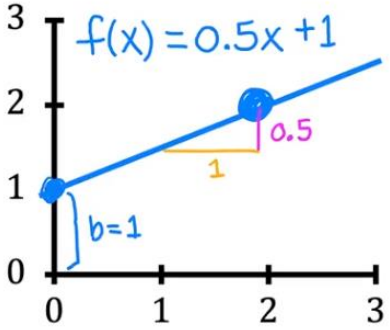
w, b: parameters
coefficients
weights



→ $w = 0$
→ $b = 1.5$ (y-intercept)



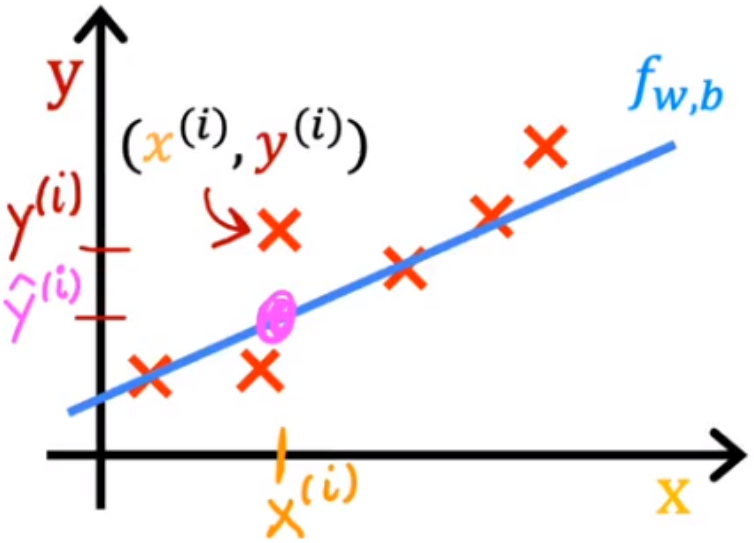
→ $w = 0.5$
→ $b = 0$



→ $w = 0.5$
→ $b = 1$

1. Regression – cost function

Cost function: Squared error cost function



$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

$\hat{y}^{(i)}$: error

m = number of training examples

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

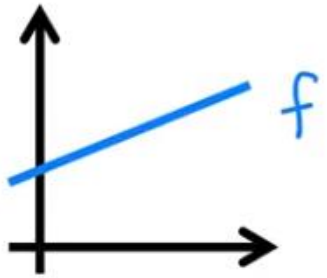
1. Regression – cost function

model:

$$f_{w,b}(x) = wx + b$$

parameters:

w, b



cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

goal:

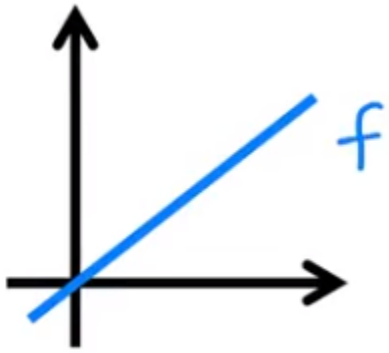
$$\underset{w,b}{\text{minimize}}\, J(w,b)$$

simplified

$$f_w(x) = wx$$

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2$$

$$\underset{w}{\text{minimize}}\, J(w)$$

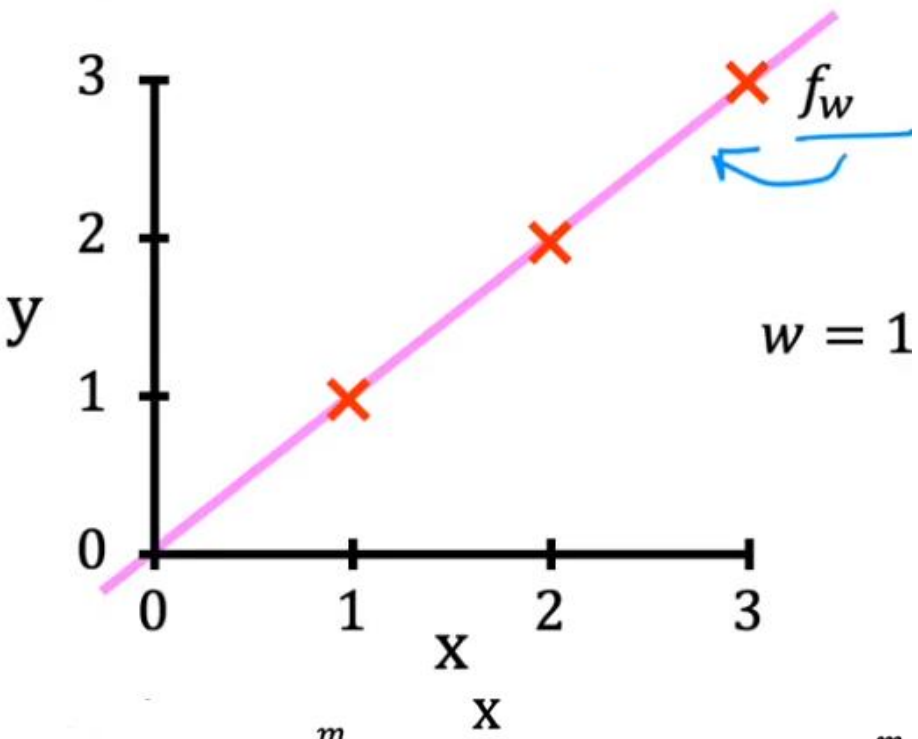


1. Regression – cost function

$f_w(x)$

(for fixed w, function of x)

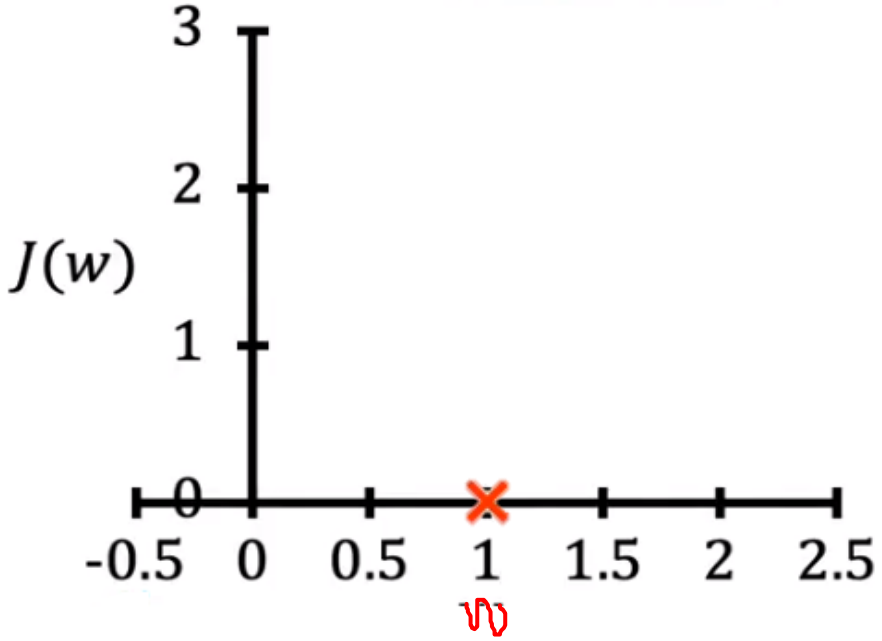
input



$J(w)$

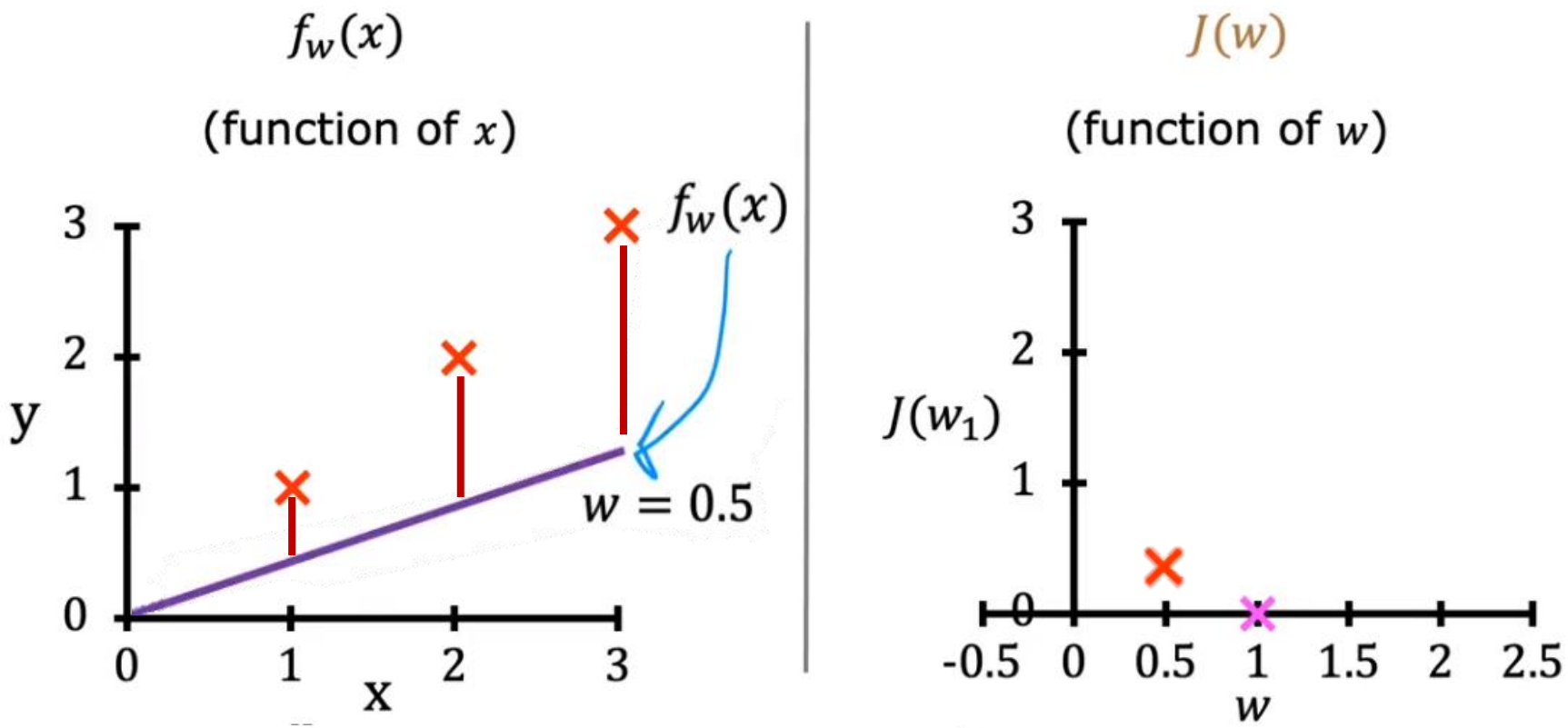
(function of w)

parameter



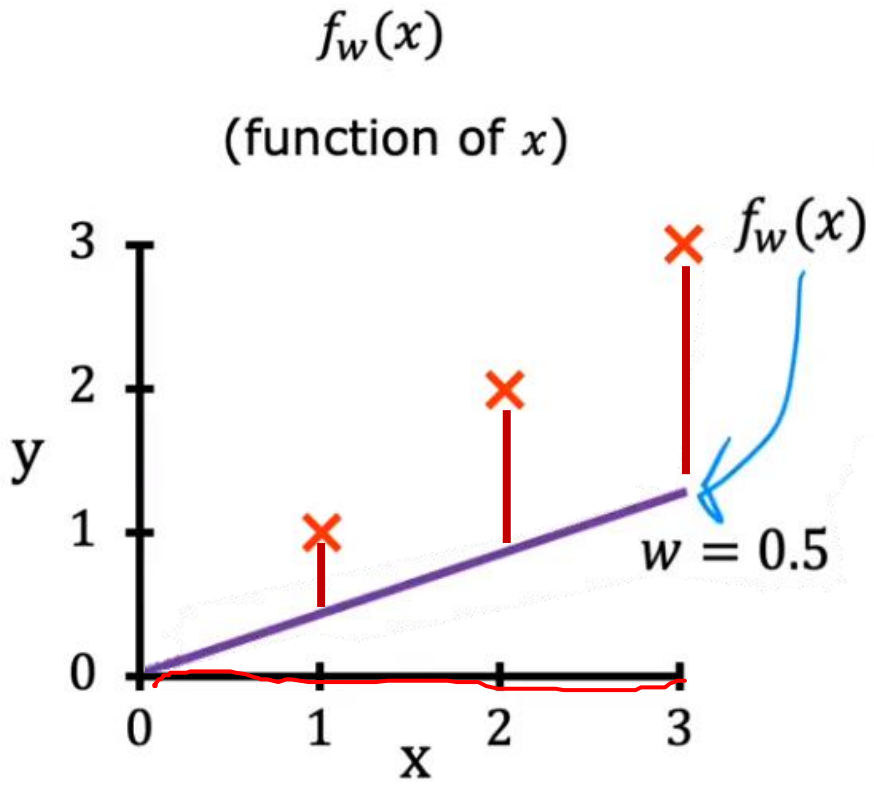
$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m}\sum_{i=1}^{m}(wx^{(i)} - y^{(i)})^2 = \frac{1}{2m}(0^2 + 0^2 + 0^2) = 0$$

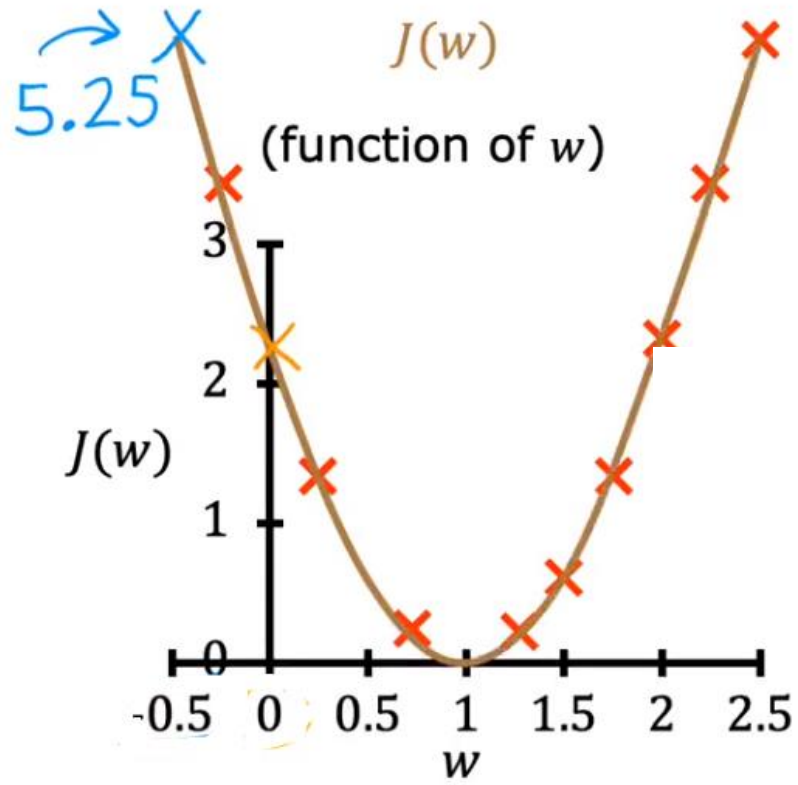
1. Regression – cost function



$J(0.5) = \frac{1}{2m}\left[(0.5-1)^2+(1-2)^2+(1.5-3)^2\right] = \frac{1}{2\times 3}[3.5] = \frac{3.5}{6} \approx 0.58$

1. Regression – cost function





$J(0) = \frac{1}{2m}(1^2 + 2^2 + 3^2) = \frac{1}{6}[14] \approx 2.3$

choose w to minimize J(w)

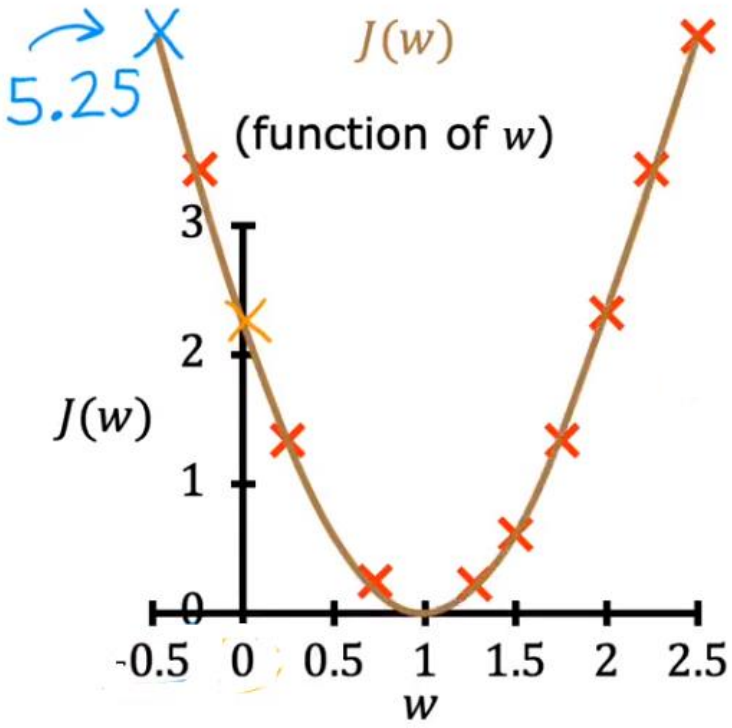
1. Regression – cost function

goal of linear regression:

$$\underset{w}{\text{minimize}}\, J(w)$$

general case:

$$\underset{w,b}{\text{minimize}}\, J(w,b)$$



1. Regression – cost function

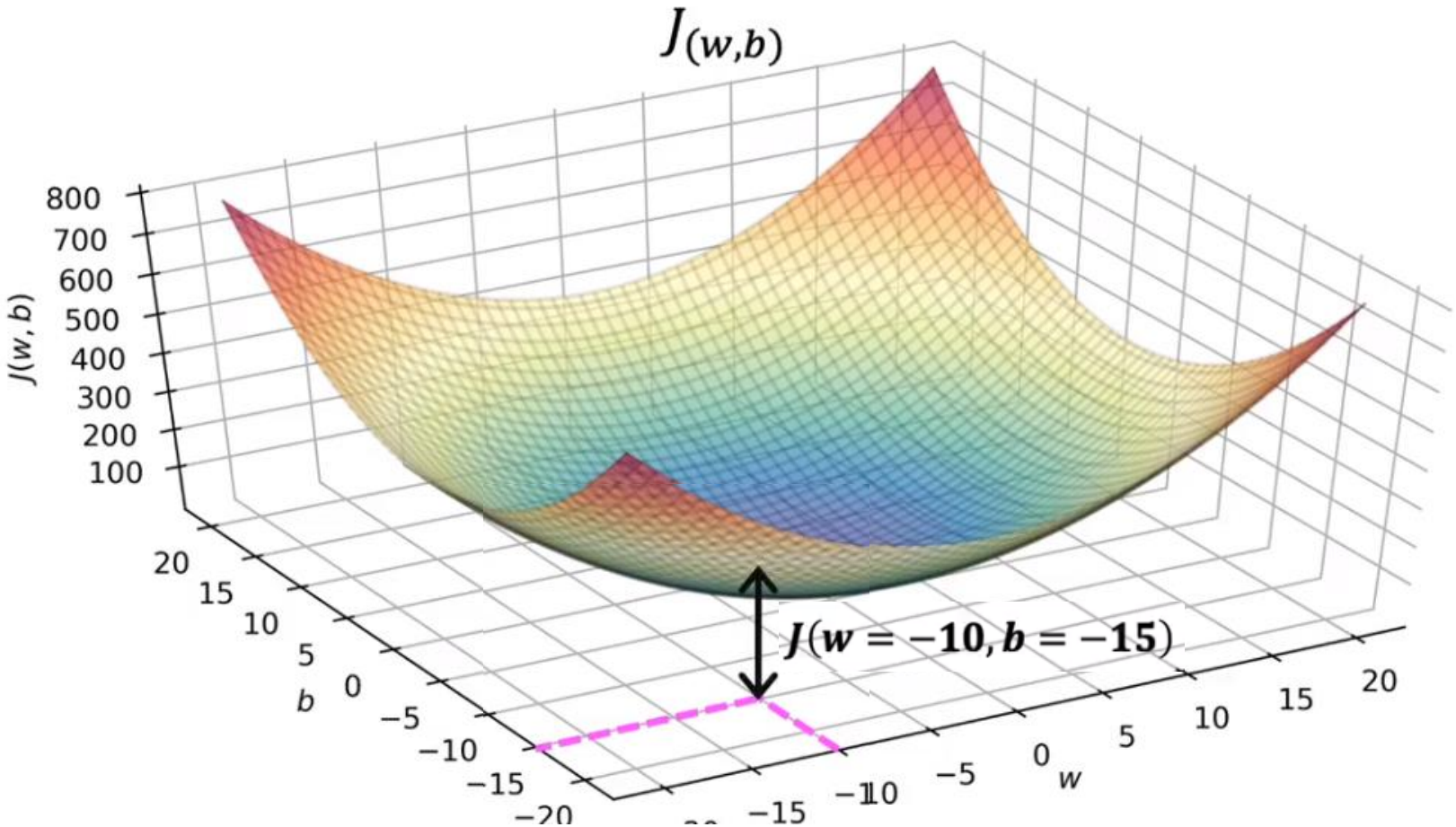
Model $f_{w,b}(x) = wx + b$

Parameters w, b

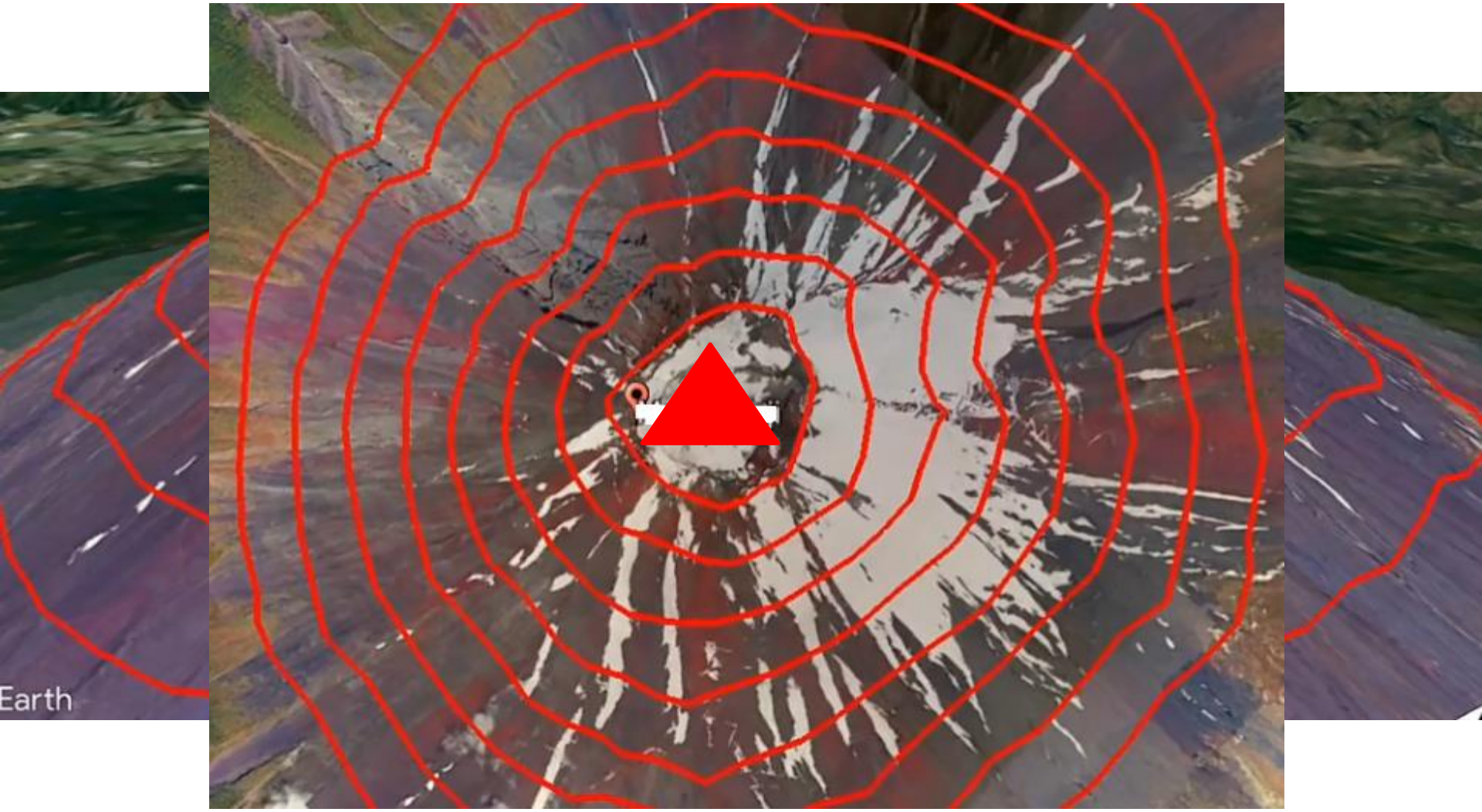
Cost Function $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$

Objective $\underset{w,b}{\text{minimize}}\, J(w,b)$

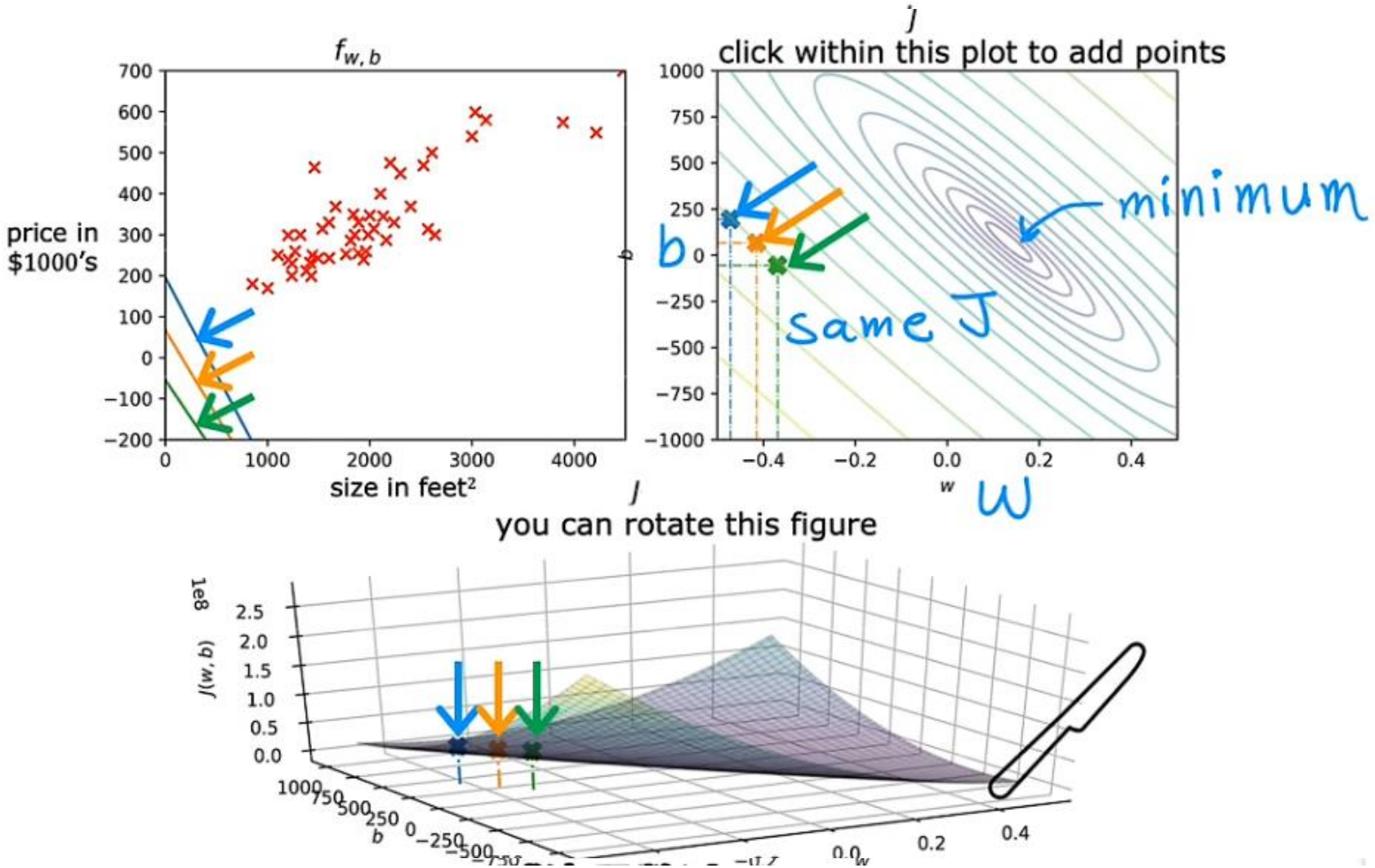
1. Regression – cost function



1. Regression – cost function



1. Regression – cost function



1. Regression – gradient descent

Have some function $J(w,b)$ for linear regression or any function

Want $\min_{w,b} J(w,b)$

$\min_{w_1,\ldots,w_n,b} J(w_1,w_2,\ldots,w_n,b)$

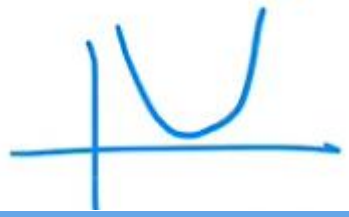
Outline:

Start with some w,b (set $w=0$, $b=0$)

Keep changing w,b to reduce $J(w,b)$

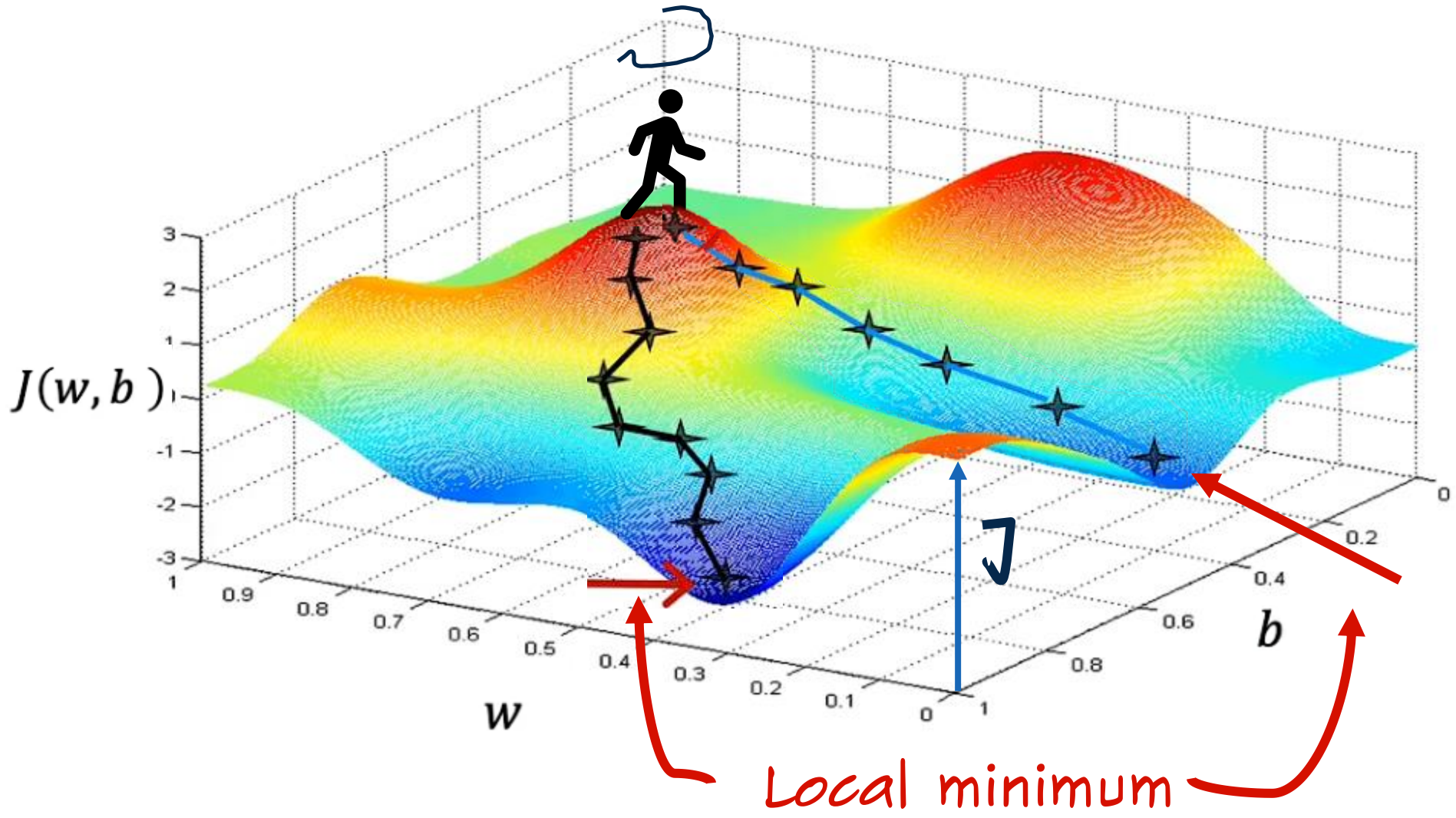
Until we settle at or near a minimum

J not always



may have >1 minimum

1. Regression – gradient descent



1. Regression – gradient decent

Gradient descent algorithm

$$w = w - \alpha \frac{d}{dw} J(w,b)$$

$$b = b - \alpha \frac{d}{db} J(w,b)$$

Learning rate

Derivative

Simultaneously update w and b

Assignment

$a = c$

$a = a + 1$

Code

Truth assertion

$a = c$

$a = a + 1$ ✗

Math

a==c

Repeat until convergence

Correct: Simultaneous update

$$tmp\_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$tmp\_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$w = tmp\_w$$

$$b = tmp\_b$$

Incorrect

$$tmp\_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$w = tmp\_w$$

$$tmp\_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$b = tmp\_b$$

1. Regression – gradient decent

Gradient descent algorithm

repeat until convergence {

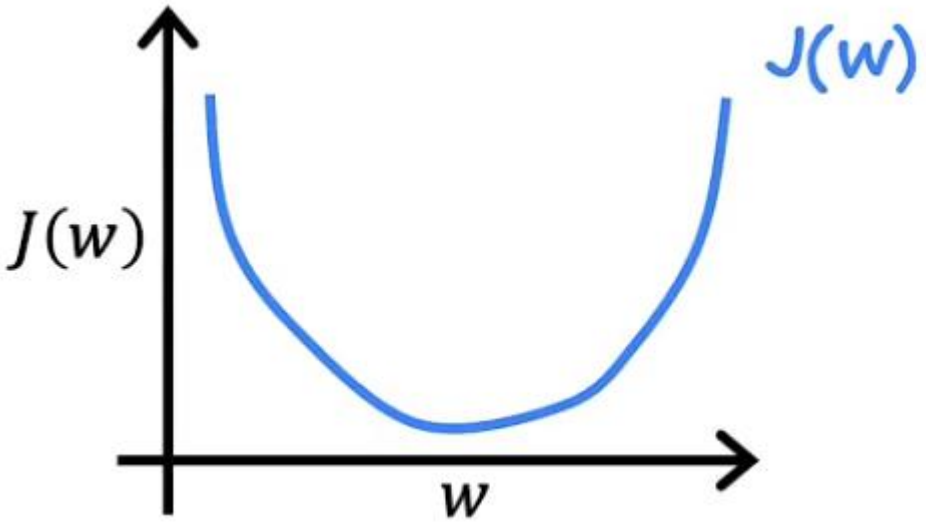
$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

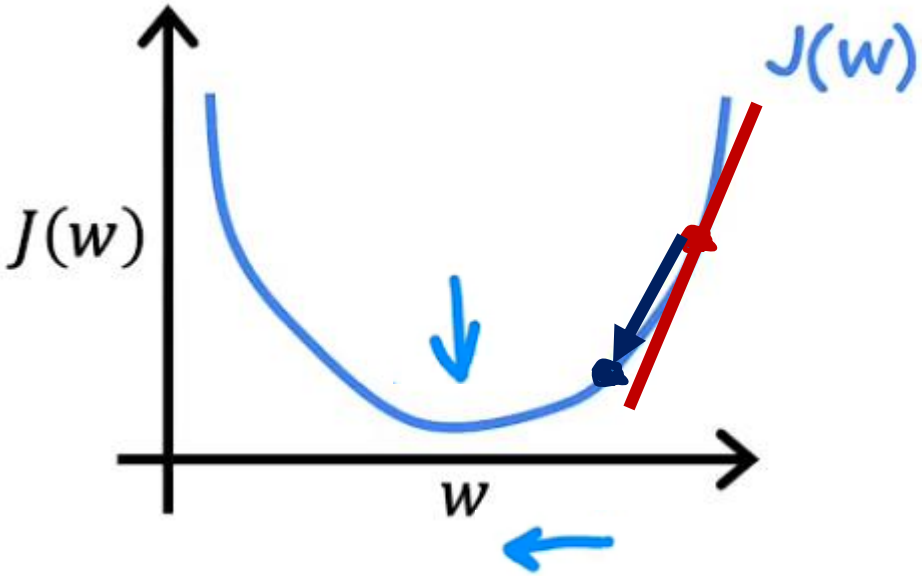
$J(w)$

$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

$$\min_w J(w)$$



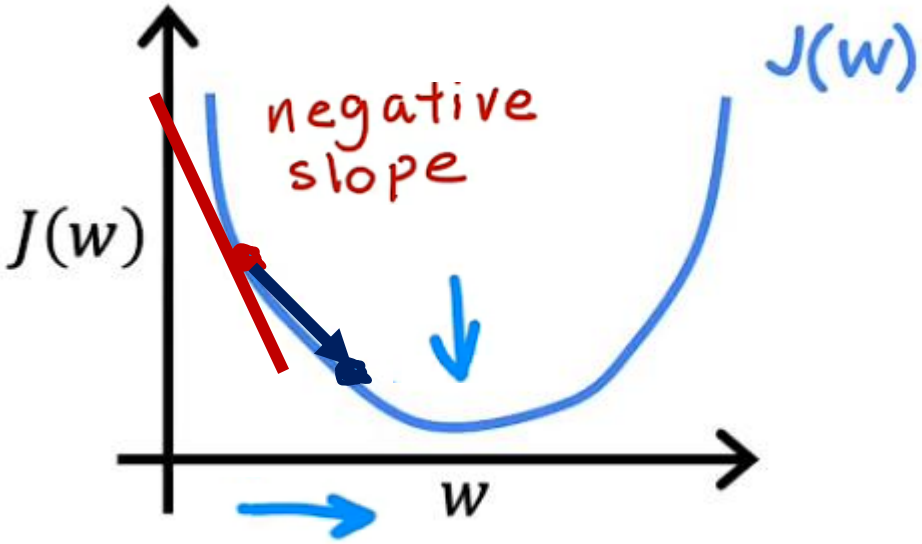
1. Regression – gradient decent



$$w = w - \alpha \frac{d}{dw} J(w)$$

$$> 0$$

$$w = w - \alpha \cdot (positive\ number)$$



$$\frac{d}{dw} J(w) < 0$$

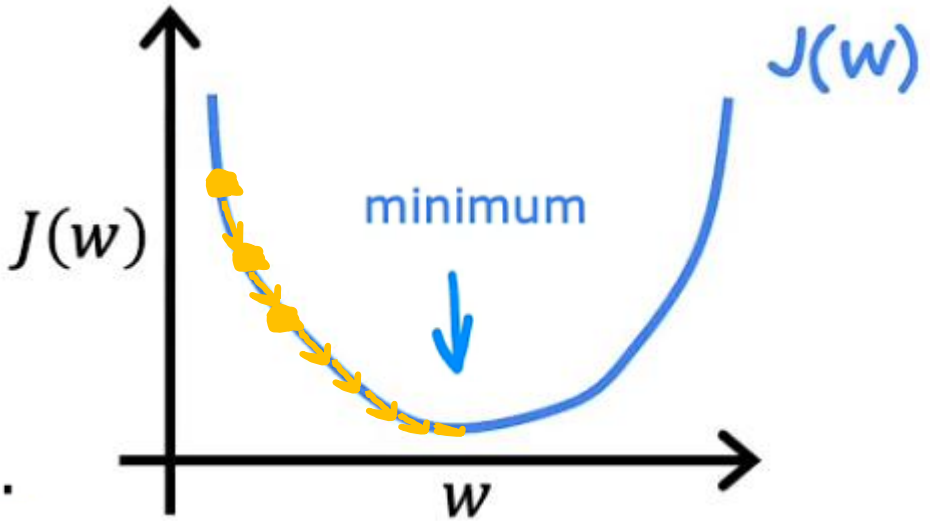
$$w = w - \alpha \cdot (negative\ number)$$

1. Regression – gradient decent

$$w = w - \alpha \frac{d}{dw} J(w)$$

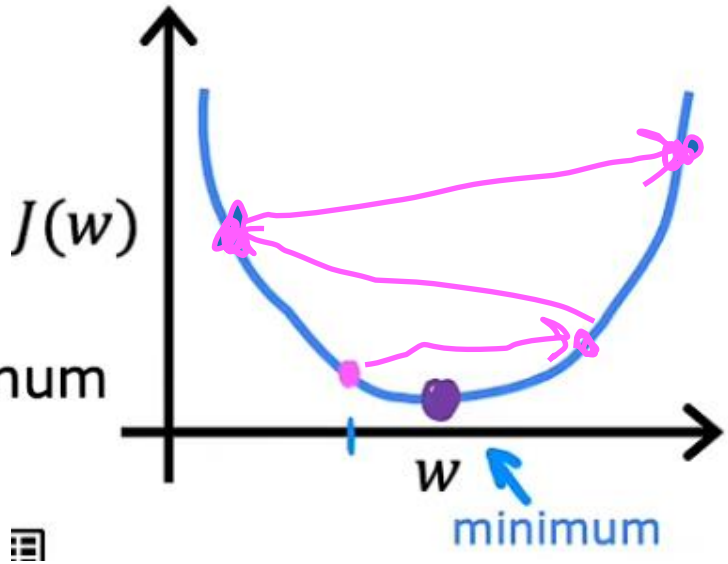
If α is too small...

Gradient descent may be slow.

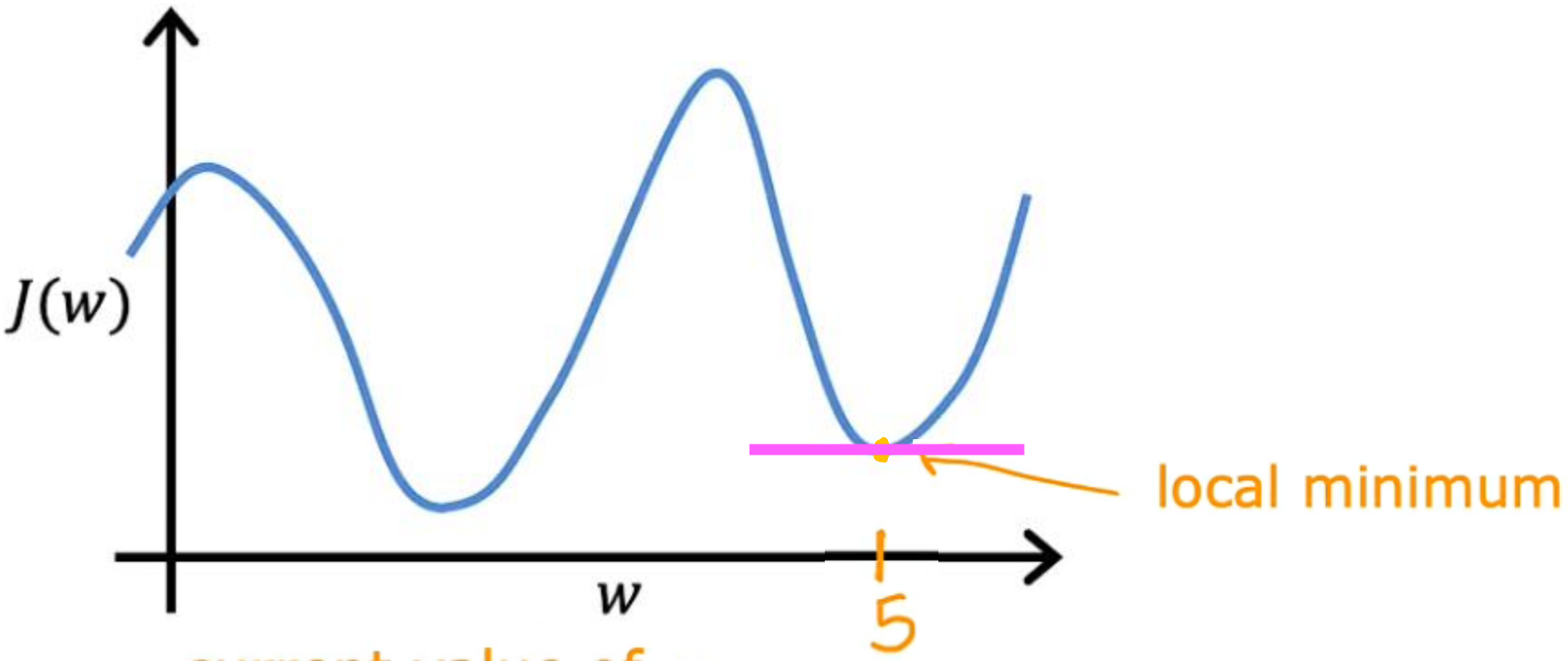


If α is too large...

Gradient descent may:
- Overshoot, never reach minimum



1. Regression – gradient decent



current value of w

$$w = w - \alpha \frac{d}{dw} J(w)$$

$$w = w - \alpha \cdot 0$$

$$w = w$$

1. Regression – gradient decent

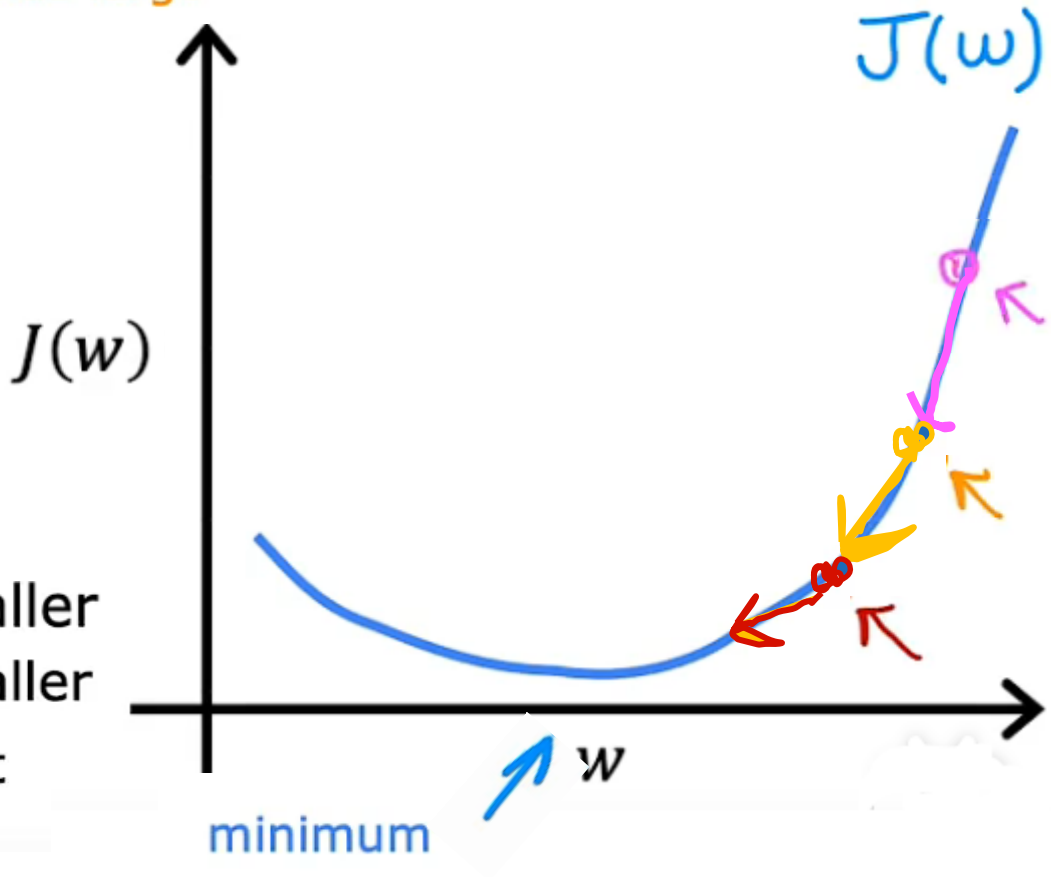
Can reach local minimum with fixed learning rate

smaller

not as large

$$w = w - \alpha \frac{d}{dw} J(w)$$

large



Near a local minimum,

- Derivative becomes smaller
- Update steps become smaller

Can reach minimum without decreasing learning rate

1. Regression – gradient decent

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$$

}

1. Regression – gradient decent

(Optional)

$$\frac{\partial}{\partial w} J(w,b) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) 2x^{(i)} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right) x^{(i)}$$

$$\frac{\partial}{\partial b} J(w,b) = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) 2 = \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)$$

1. Regression – gradient decent

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) \quad x^{(i)}$$

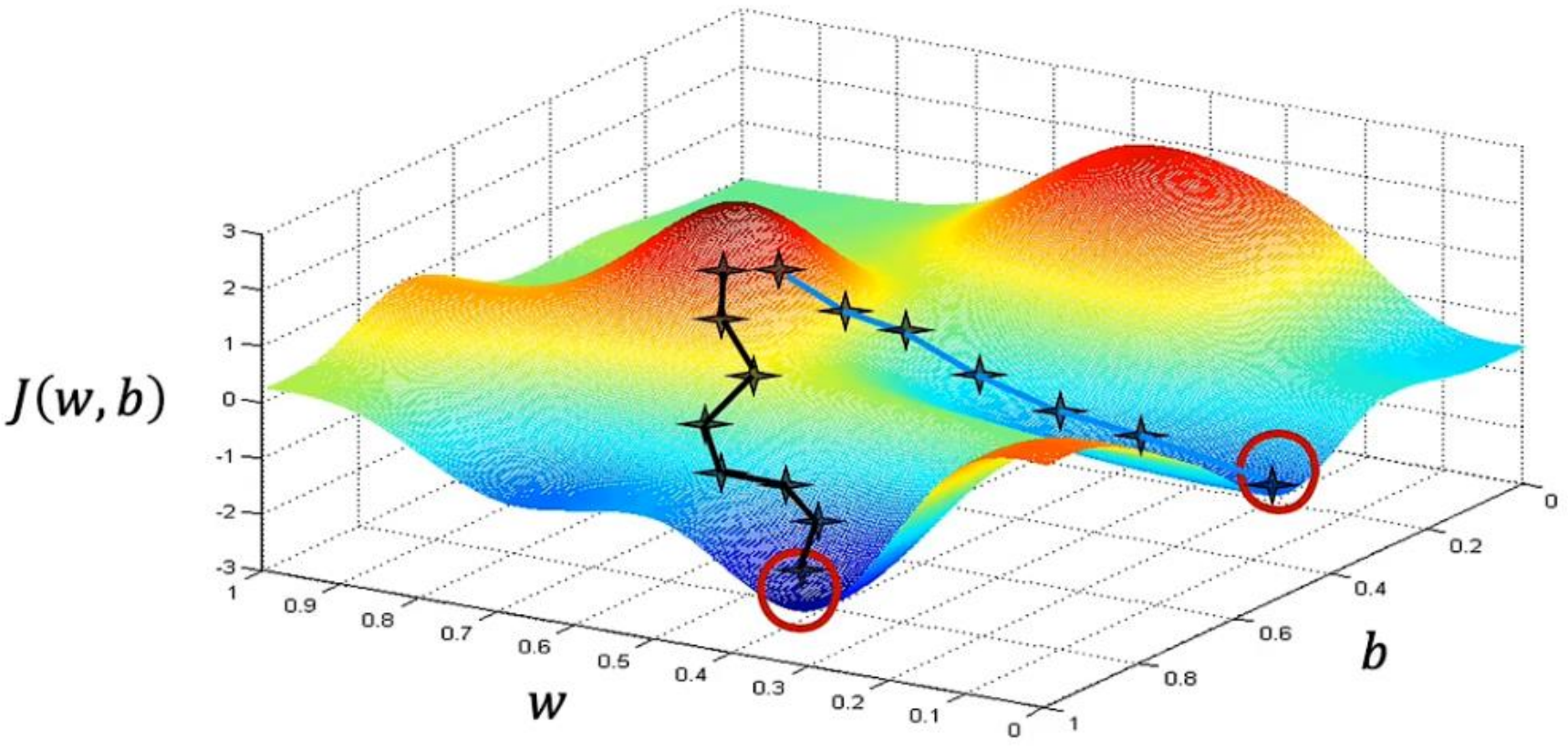
$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

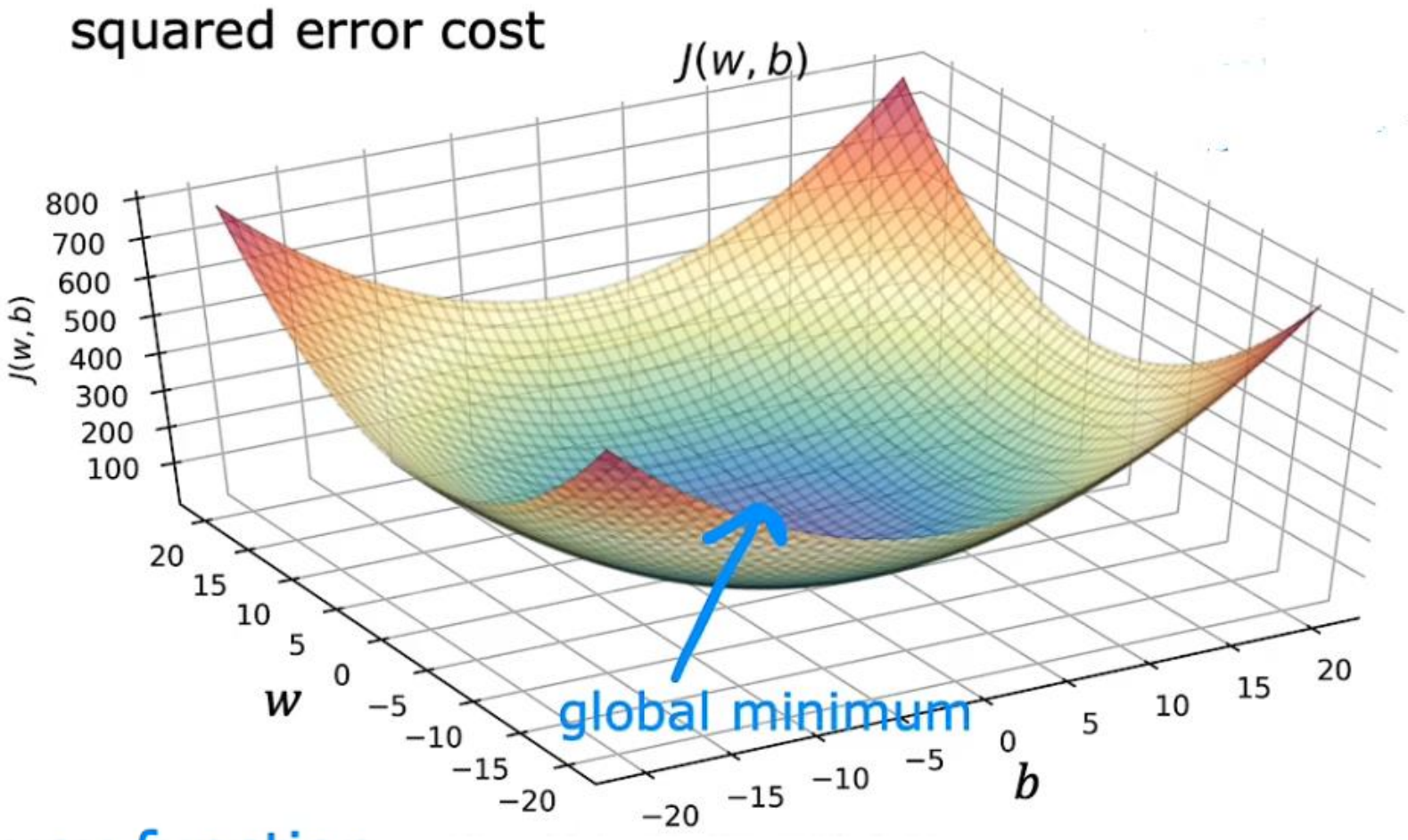
Update w and b simultaneously

1. Regression – gradient decent

More than one local minimum

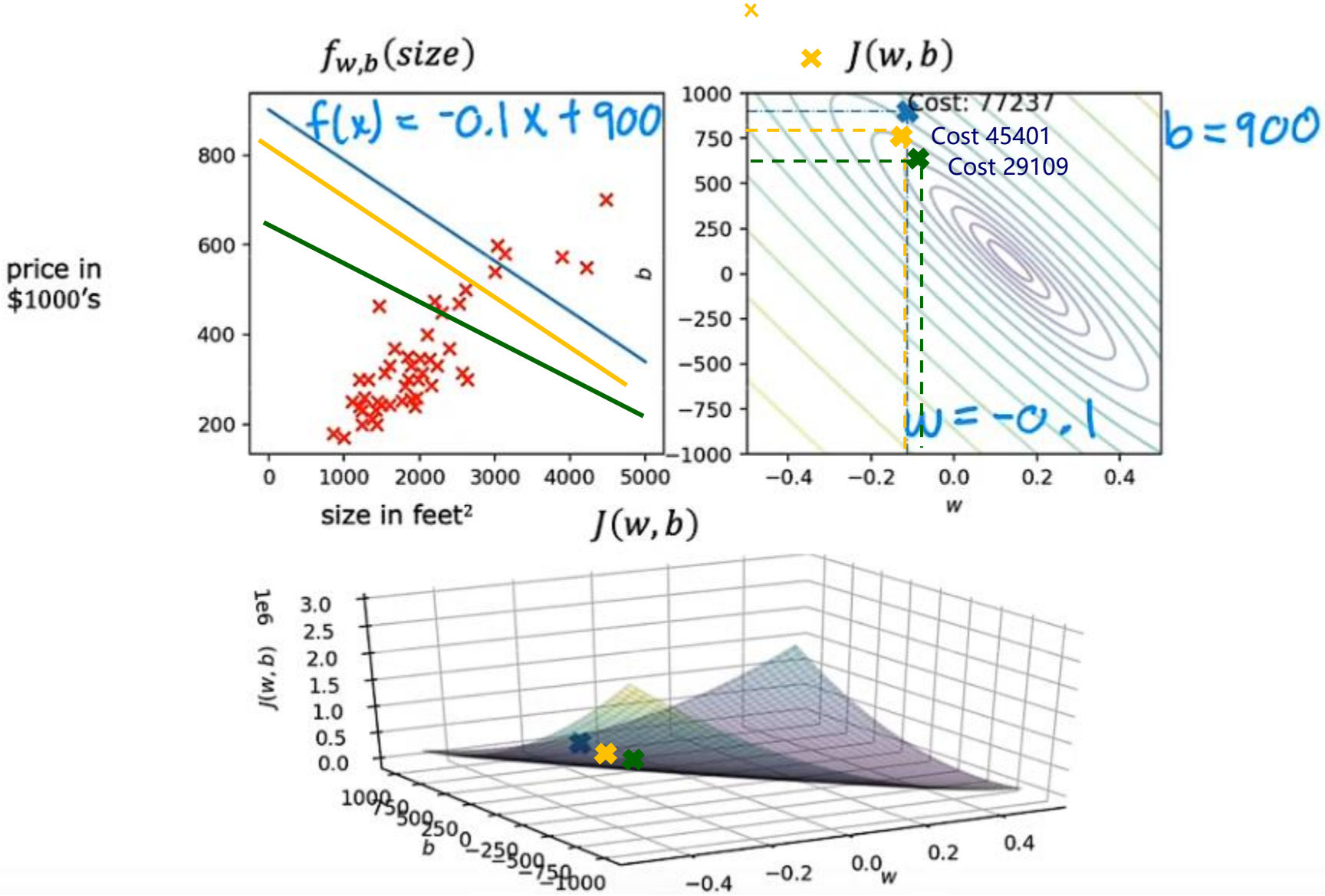


1. Regression – gradient decent

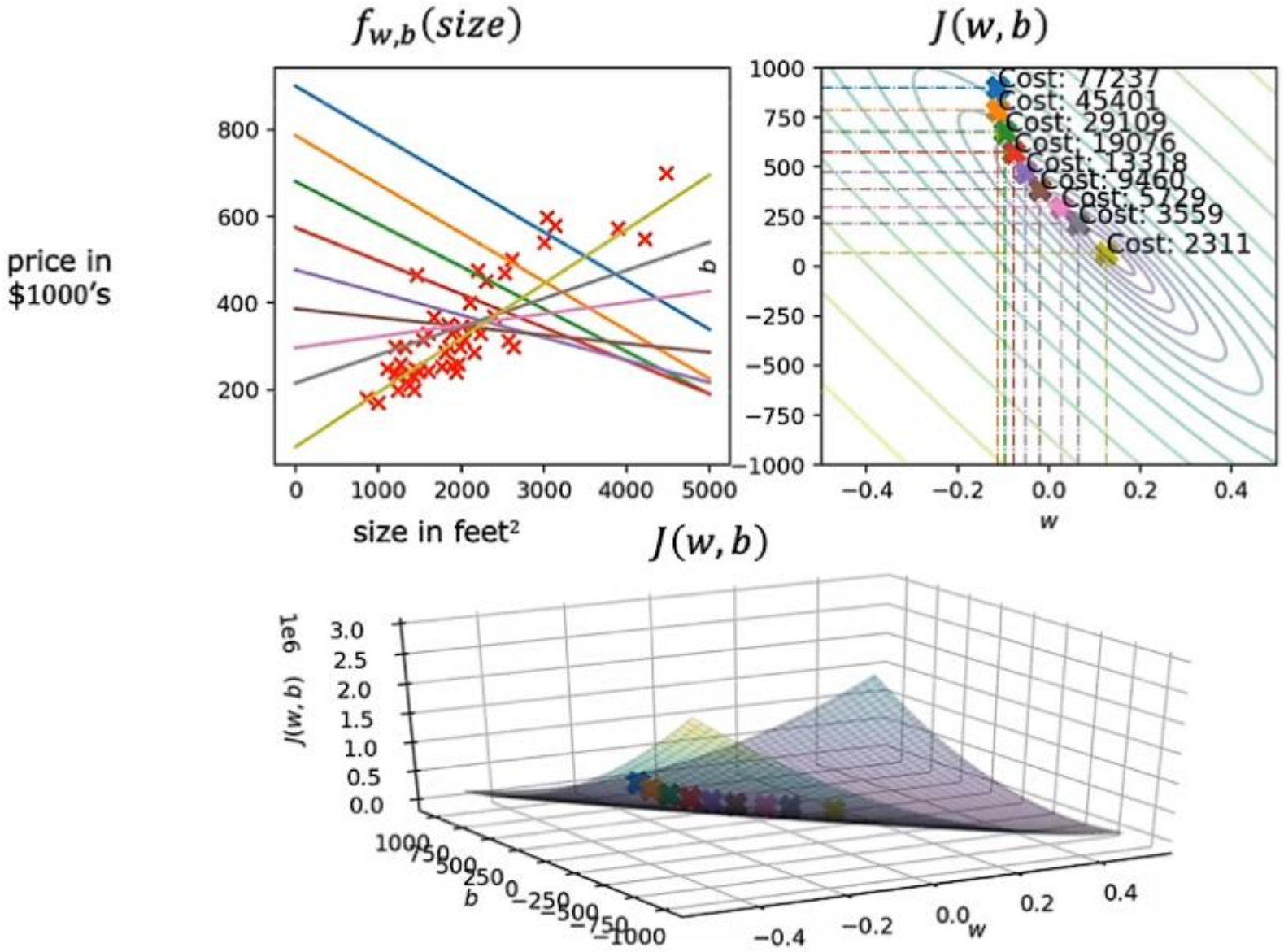


convex function

1. Regression – gradient decent



1. Regression – gradient decent



1. Regression – gradient decent

"Batch" gradient descent

"Batch": Each step of gradient descent uses all the training examples.

$$\sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

2. Multiple features

Multiple features (variables)

one feature →

Size in feet² (x)	Price ($) in 1000's ($y$)
2104	400
1416	232
1534	315
852	178
...	...

←

$$f_{w,b}(x) = wx + b$$

2. Multiple features

Multiple features (variables)

Size in feet² X_1	Number of bedrooms X_2	Number of floors X_3	Age of home in years X_4	Price ($) in $1000's
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

x_j = j^{th} feature

n = number of features

$\vec{x}^{(i)}$ = features of i^{th} training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

$\vec{x}^{(2)} = [1416 \quad 3 \quad 2 \quad 40]$

$x_3^{(2)} = 2$

2. Multiple features

Model:

Previously: $f_{w,b}(x) = wx + b$

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

$$f_{w,b}(x) = 0.1 x_1 + 4 x_2 + 10 x_3 + -2 x_4 + 80$$

x_1: size

x_2: #bedrooms

x_3: #floors

x_4: years

80: base price

2. Multiple features

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

$\vec{w} = [w_1 \; w_2 \; w_3 \ldots w_n]$

b is a number

parameters of the model

$\vec{x} = [x_1 \; x_2 \; x_3 \ldots x_n]$

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

dot product

multiple linear regression

2. Multiple features

Parameters and features

$\vec{w} = [w_1 \quad w_2 \quad w_3]$

b is a number

$\vec{x} = [x_1 \quad x_2 \quad x_3]$

linear algebra: count from 1

NumPy

```
w = np.array([1.0,2.5,-3.3])
b = 4
x = np.array([10,20,30])
```

x[0] x[1] x[2]

w[0] w[1] w[2]

code: count from 0

Without vectorization

$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$

```
f = w[0] * x[0] +
    w[1] * x[1] +
    w[2] * x[2] + b
```

Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \sum_{j=1}^{n} w_j x_j + b$$

```
f = 0
for j in range(0,n):
    f = f + w[j] * x[j]
f = f + b
```

range(0,n) → j = 0...n-1

Vectorization

$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

```
f = np.dot(w,x) + b
```

2. Multiple features

Without vectorization

```
for j in range(0,16):
    f = f + w[j] * x[j]
```

t_0

```
f + w[0] * x[0]
```

t_1

```
f + w[1] * x[1]
```

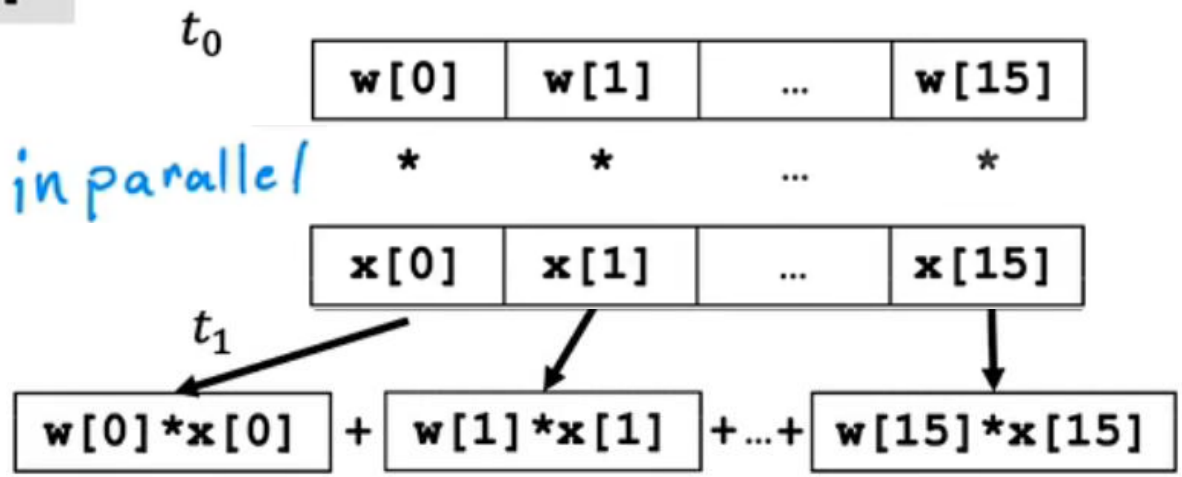
...

t_{15}

```
f + w[15] * x[15]
```

Vectorization

```
np.dot(w,x)
```



efficient → scale to large datasets

2. Multiple features

Gradient descent $\vec{w} = (w_1 \quad w_2 \quad \cdots \quad w_{16})$ ~~b~~ parameters

derivatives $\vec{d} = (d_1 \quad d_2 \quad \cdots \quad d_{16})$

```
w = np.array([0.5, 1.3, … 3.4])
d = np.array([0.3, 0.2, … 0.4])
```

compute $w_j = w_j - 0.1 d_j$ for $j = 1 \ldots 16$ (learning rate α)

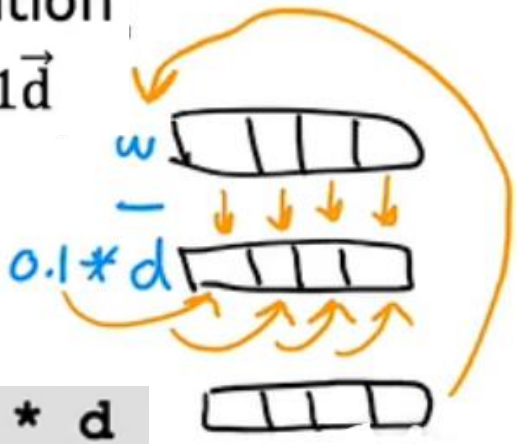
Without vectorization

$$w_1 = w_1 - 0.1 d_1$$
$$w_2 = w_2 - 0.1 d_2$$
$$\vdots$$
$$w_{16} = w_{16} - 0.1 d_{16}$$

```
for j in range(0,16):
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

$$\vec{w} = \vec{w} - 0.1\vec{d}$$




```
w = w - 0.1 * d
```

2. Multiple features

	Previous notation	Vector notation
Parameters	$w_1, \cdots, w_n$ b	$\vec{w} = [w_1 \quad \cdots \quad w_n]$ ← vector of length n b still a number
Model	$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \cdots + w_n x_n + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ (· dot product)
Cost function	$J(w_1, \cdots, w_n, b)$	$J(\vec{w}, b)$

Gradient descent

repeat {

$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \cdots, w_n, b)$

$b = b - \alpha \frac{\partial}{\partial b} J(w_1, \cdots, w_n, b)$

}

repeat {

$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$

$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$

}

3. Feature scaling

Feature and parameter values

$\widehat{price} = w_1 x_1 + w_2 x_2 + b$

size #bedrooms

x_1: size (feet^2)
range: 300 – 2,000

x_2: # bedrooms
range: 0 – 5

large Small

House: $x_1 = 2000$, $x_2 = 5$, $price = \$500k$

size of the parameters w_1, w_2?

$w_1 = 50$, $w_2 = 0.1$, $b = 50$

$w_1 = 0.1$, $w_2 = 50$, $b = 50$
small large

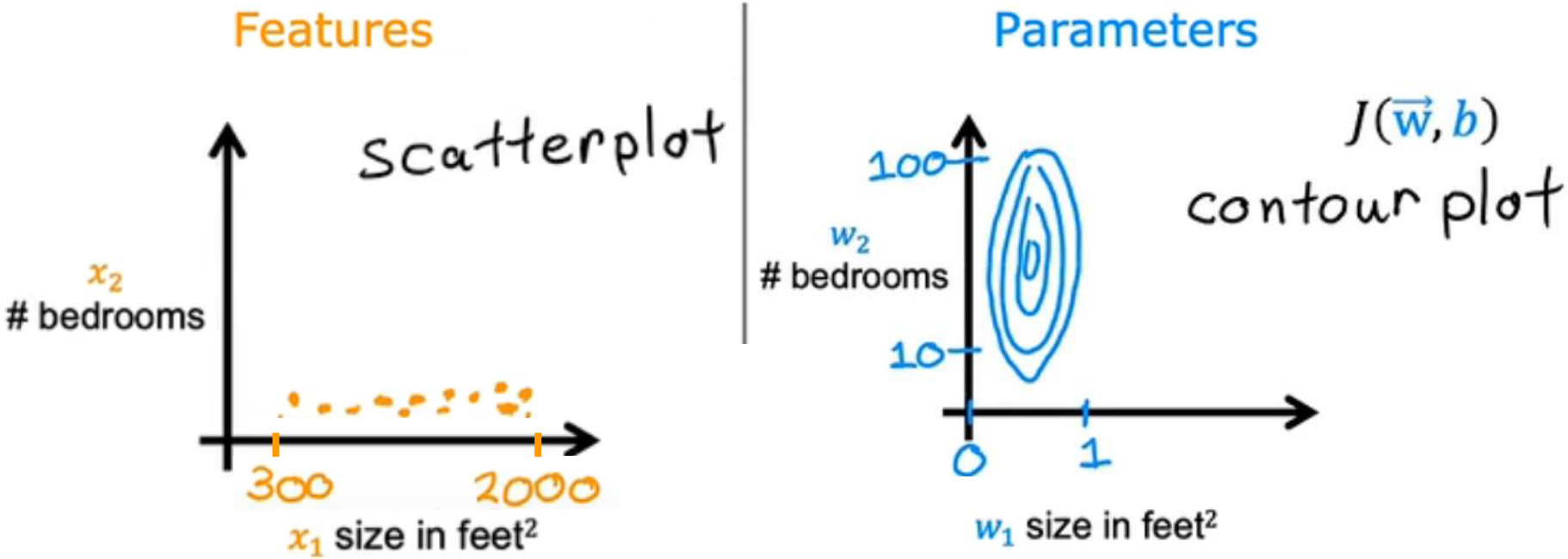
$\widehat{price} = \underbrace{50 * 2000}_{100{,}000\,k} + \underbrace{0.1 * 5}_{0.5\,k} + \underbrace{50}_{50k}$

$\widehat{price} = \underbrace{0.1 * 2000k}_{200k} + \underbrace{50 * 5}_{250\,k} + \underbrace{50}_{50k}$

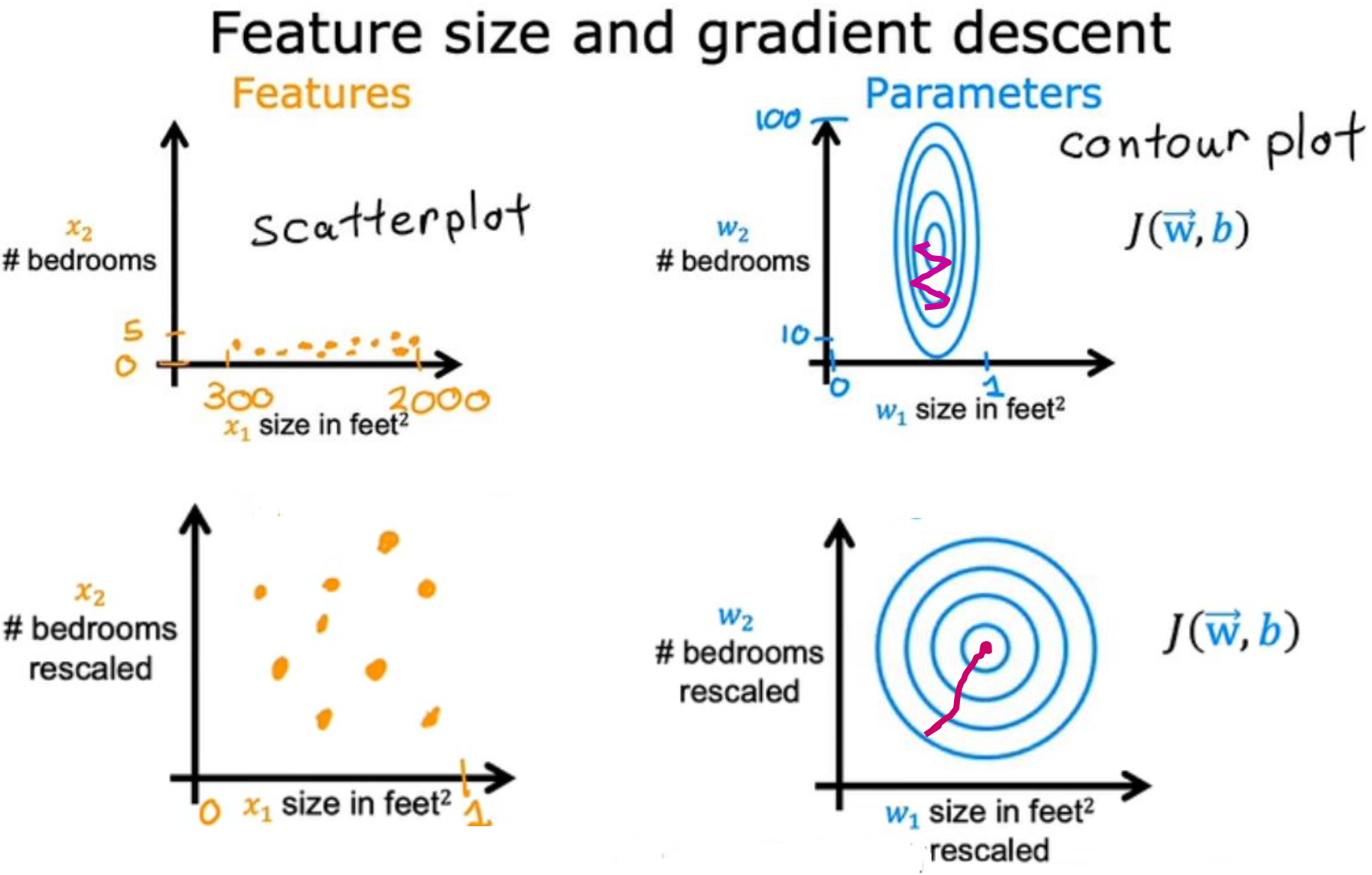
3. Feature scaling

Feature size and parameter size

	size of feature x_j	size of parameter w_j
size in feet^2	⟷	⟷
#bedrooms	⟷	⟷

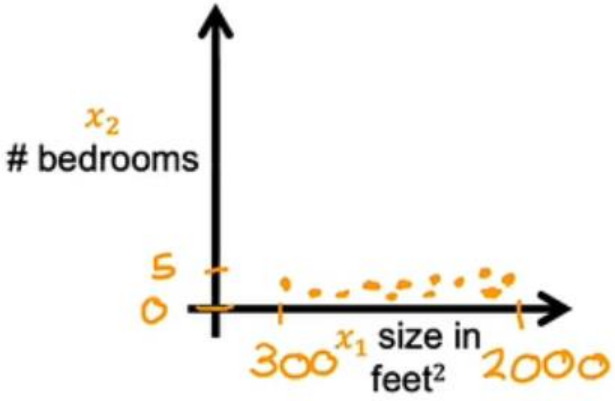


3. Feature scaling



3. Feature scaling

Feature scaling



$$300 \le x_1 \le 2000$$

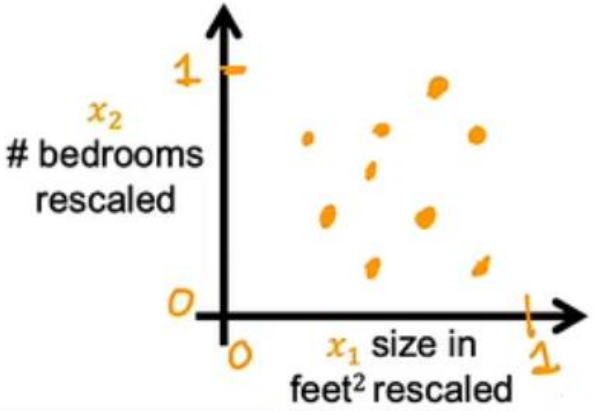
$$x_{1,scaled} = \frac{x_1}{2000}$$

max

$$0 \le x_2 \le 5$$

$$x_{2,scaled} = \frac{x_2}{5}$$

max

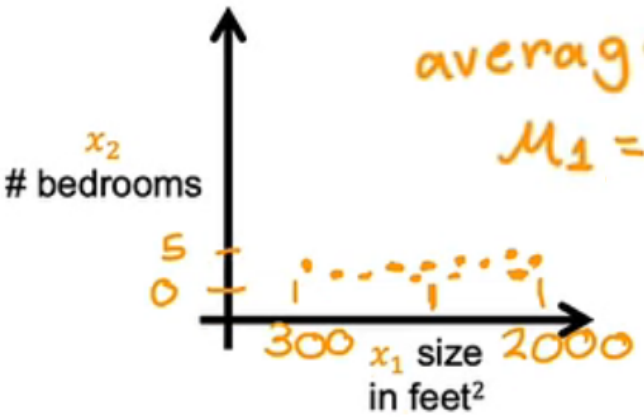


$$0.15 \le x_{1,scaled} \le 1$$

$$0 \le x_{2,scaled} \le 1$$

3. Feature scaling

Mean normalization



average

$\mu_1 = 600$

$\mu_2 = 2.3$

$300 \le x_1 \le 2000$

$0 \le x_2 \le 5$

$$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$$

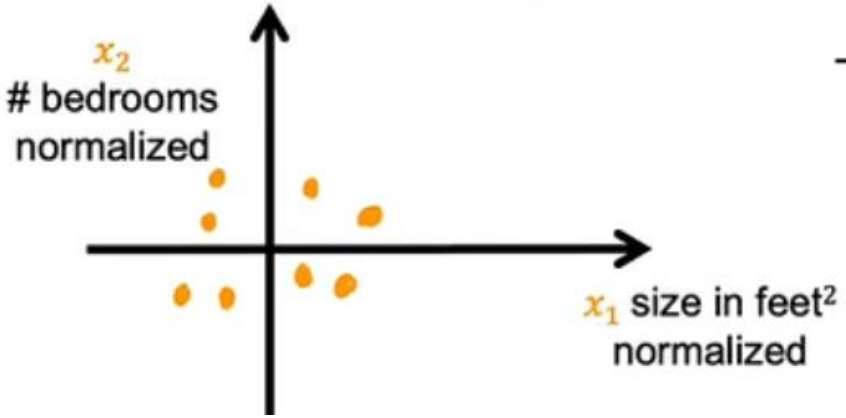
max - min

$$x_2 = \frac{x_2 - \mu_2}{5 - 0}$$

max - min

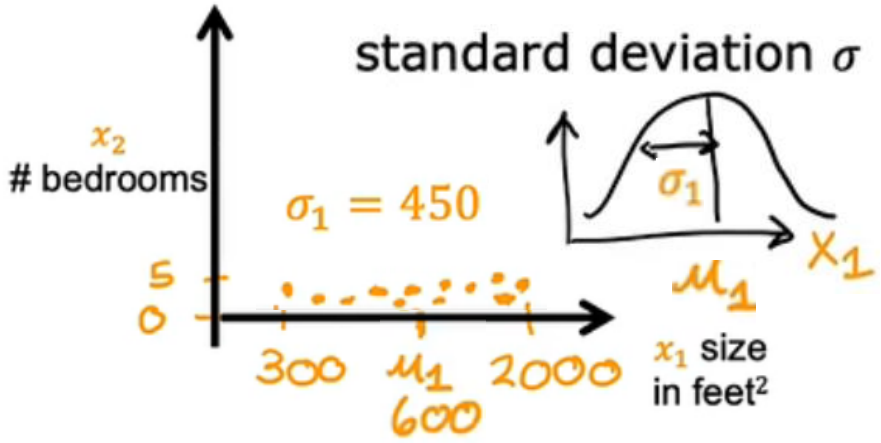
$-0.18 \le x_1 \le 0.82$

$-0.46 \le x_2 \le 0.54$



3. Feature scaling

Z-score normalization



$300 \leq x_1 \leq 2000$

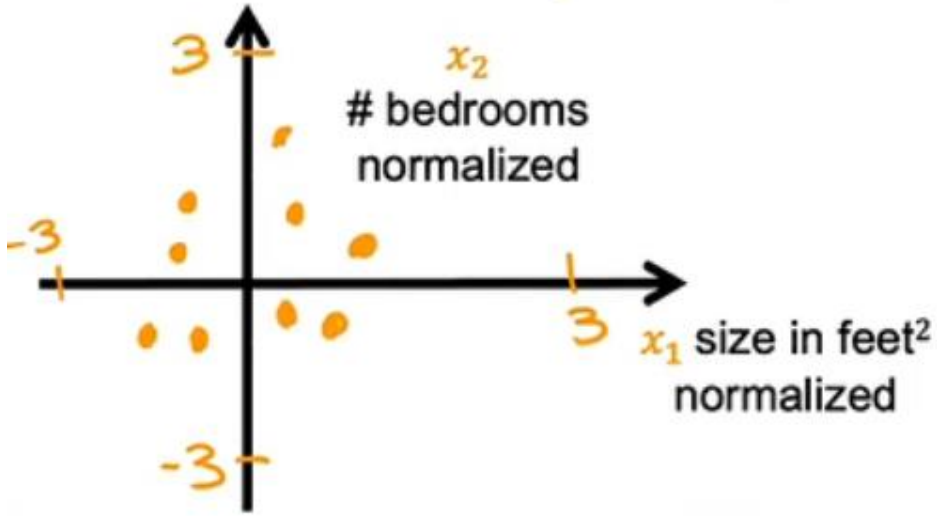
$$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$$

$-0.67 \leq x_1 \leq 3.1$

$0 \leq x_2 \leq 5$

$$x_2 = \frac{x_2 - \mu_2}{\sigma_2}$$

$-1.6 \leq x_2 \leq 1.9$



3. Feature scaling

Feature scaling

aim for about $-1 \leq x_j \leq 1$ for each feature x_j

$-3 \leq x_j \leq 3$

$-0.3 \leq x_j \leq 0.3$

} acceptable ranges

$0 \leq x_1 \leq 3$ okay, no rescaling

$-2 \leq x_2 \leq 0.5$

$-100 \leq x_3 \leq 100$ too large → rescale

$-0.001 \leq x_4 \leq 0.001$ too small → rescale

4. Convergence

Gradient descent

$$\begin{cases} w_j = w_j - \alpha \dfrac{\partial}{\partial w_j} J(\vec{w}, b) \\ b = b - \alpha \dfrac{\partial}{\partial b} J(\vec{w}, b) \end{cases}$$

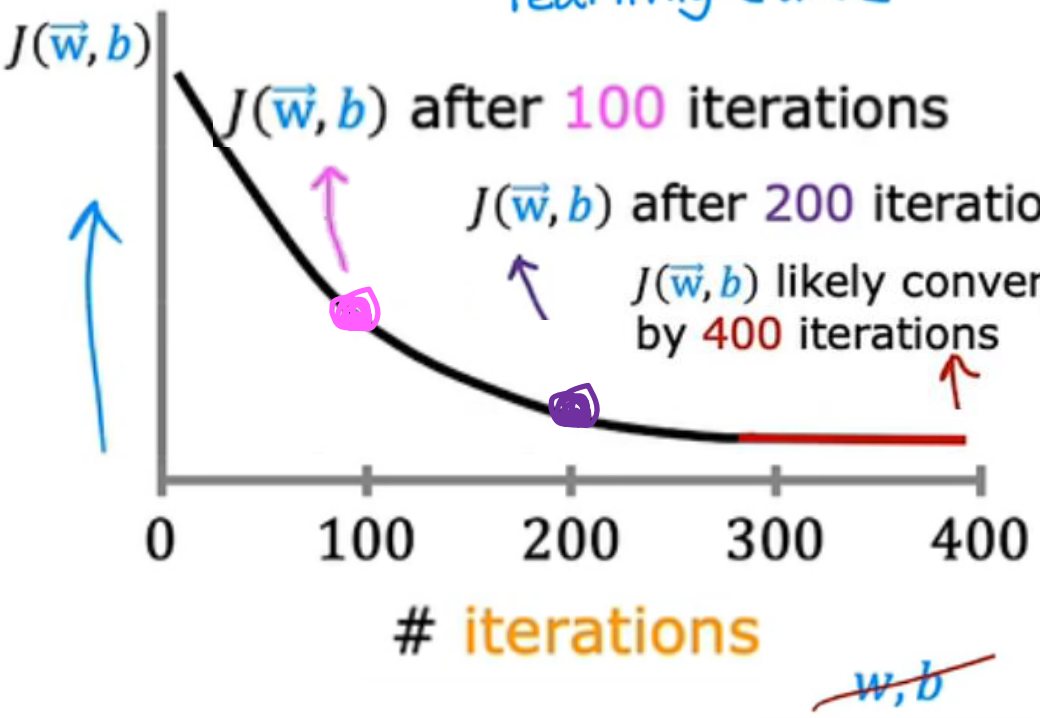
4. Convergence

Make sure gradient descent is working correctly

objective: $\min_{\vec{w},b} J(\vec{w},b)$

$J(\vec{w},b)$ should decrease after every iteration

learning curve



~~w,b~~

Automatic convergence test

Let ε "epsilon" be 10^{-3}.

0.001

If $J(\vec{w},b)$ decreases by $\leq \varepsilon$ in one iteration, declare convergence.

(found parameters $\vec{w},b$ to get close to global minimum)

iterations needed varies

BIT.MLMR

End of Lecture 2

Have you grasp the main ideas and methods of supervised and unsupervised learning?